

Instituto Federal de Educação, Ciência e
Tecnologia de São Paulo campus Votuporanga

Apostila da disciplina Desenvolvimento Web 3 (DW3)

Prof. Dr. Evandro Jardini
eajardini@ifsp.edu.br

Esta apostila possui conteúdo extraído de diversos materiais como livros, apostilas, notas, artigos, etc elencados nas de Referências Bibliografias.

Sumário

1 Computação Distribuída utilizando o Formato *JavaScript Object Notation* (JSON) 9

1.1 Introdução 9

1.2 Computação Distribuída 10

1.2.1 Vantagens da Computação Distribuída 11

1.2.2 Desvantagens da Computação Distribuída 12

1.3 *JavaScript Object Notation* (JSON) 12

1.3.1 Dados JSON 13

1.3.2 Objetos JSON 14

1.3.3 Matrizes JSON 14

1.3.4 Objetos JSON Encadeados 15

1.3.5 Acessando os dados de uma objeto JSON 16

1.4 *WebServices* 18

1.5 *Application Programming Interface* (API) e *Representational State Transfer* (REST) 19

1.5.1 Exemplo de API REST 21
1.5.1.1 Exercício 22
1.5.1.2 Segurança em *API Rest* 23
1.6 Computadores *Back-End* e *Front-End* 25
1.6.1 Computador *Back-End* 25
1.6.2 Computador *Front-End* 25
1.7 Finalizando 26

2 Implementação do Servidor *Back-End* 27
2.1 Introdução 27
2.1.1 Diretório padrão da disciplina 28
2.1.2 Repositório Remoto no *Github* da disciplina 28
2.2 *Node.js* 29
2.2.1 Instalação do Node.js 30
2.2.2 Criando a primeira aplicação em Node.js 31
2.2.3 Criando a segunda aplicação em Node.js 34
2.2.4 Exercícios 44
2.3 Servidor *Back-End do* Sistema Acadêmico 45
2.3.1 Banco de dados 45
2.3.2 Procedimentos iniciais 49
2.3.3 Configurando o Cross-Origin Resource Sharing (CORS) 53
2.3.3.1 Alterando o *app.js* para configurar o CORS 54

2.3.4 Configurando o arquivo de rotas . . . 56
2.3.5 Configurando o banco de dados . . . 60
2.3.6 Configurando o diretório de aplicação: *apps* . . . 61
2.3.7 Considerações sobre a implementação dos módulos do sistema . . . 62
2.3.7.1 As operações de CRUD . . . 62
2.3.8 Configurando o módulo de *login* . . . 63
2.3.8.1 Crie os diretórios: . . . 63
2.3.8.2 Configurando o *model* . . . 63
2.3.8.3 Configurando o *controller* . . . 64
2.3.8.4 Configurando o *test* . . . 66
2.3.8.5 Configure as rotas . . . 67
2.3.8.6 Execute os testes . . . 69
2.3.9 Configurando o módulo de *aluno* . . . 69
2.3.9.1 Crie os diretórios . . . 69
2.3.9.2 Configurando o model . . . 70
2.3.9.3 Configurando o controller . . . 75
2.3.9.4 Configurando o test . . . 78
2.3.9.5 Configurando as rotas . . . 81
2.3.9.6 Execute os testes . . . 83
2.3.10 Configurando o módulo de curso . . . 83
2.3.10.1 Crie os diretórios . . . 83

2.3.10.2 Configurando o model 83
2.3.10.3 Configurando o controller 84
2.3.10.4 Configurando o test 84
2.3.10.5 Configurando as rotas 84
2.3.10.6 Execute os testes 86
2.4 Exercícios 86
2.5 Finalizando 86

3 Implementação do Servidor *Front-End* 87
3.1 Introdução 87
3.2 Configuração inicial 88
3.2.1 Instalando as bibliotecas necessárias 88
3.2.2 Instalando mais bibliotecas 90
3.2.3 Descompactando o layout 90
3.2.4 Arquivo *env* para configuração do servidor 90
3.2.5 Arquivo App.js inicial 92
3.2.6 Executando o servidor *front-end* 94
3.2.7 diretório *views* 95
3.3 Implementando as rotas *index* e a *Login* 100
3.4 Utilizando os *layouts* e a tela de *Login* 103
3.5 Implementando a lógica do *Login* 106
3.5.1 Criando a *View* de *Login* 110

3.5.2 Alterando a rota do *Login* . . . 111
3.5.3 Acrescentando verificação de usuário autenticado . . . 113
3.6 Implementando a lógica de *Home* . . . 115
3.7 Implementando a ação de *logout* do sistema . . . 117
3.8 Implementando o módulo de Alunos . . . 121
3.8.1 Criando o arquivo de **lista de *Alunos*** . . . 122
3.8.2 Criando a rota de *Alunos* . . . 123
3.8.3 Alterando o arquivo *app.js* para adicionar a rota */alunos* e outras configurações . . . 125
3.8.4 Adicionando lógica ao formulário *vwLstAlunos* . . . 128
3.8.5 Criando o arquivo *vwLstAlunos* . . . 131
3.8.6 Criando as Rotas de *Alunos* . . . 140
3.8.7 Implementando o formulário de Alunos . . . 144
3.8.8 Adicionado a rota para o formulário de Alunos . . . 148
3.8.9 Implementando o cadastro de um novo Aluno . . . 150
3.8.10 Implementando a busca de Aluno para visualizar os dados . . . 164
3.8.11 Implementando a atualização de dados do Aluno . . . 169
3.8.12 Implementando a remoção do Aluno . . . 174
3.9 Implementando o módulo de Cursos . . . 179
3.10 Finalizando . . . 180

4 Deploy da Aplicação com Containers **181**
4.1 Introdução . . . 181

4.2 Conceitos fundamentais 182
4.2.1 Container 182
4.2.2 Imagem 182
4.2.3 Docker 182
4.3 Criando a imagem da aplicação 183
4.3.1 Arquivo dockerignore 183
4.3.2 Arquivo Dockerfile do servidor *BackEnd* 184
4.3.3 Criando a imagem do servidor *BackEnd* 186
4.3.4 Executando a imagem do *BackEnd* 186
4.3.5 Arquivo Dockerfile do servidor *FrontEnd* 189
4.3.6 Criando a imagem do servidor *FrontEnd* 190
4.3.7 Executando a imagem do servidor *FrontEnd* 190
4.4 Atualizando a aplicação 191
4.5 Removendo uma imagem 192
4.6 Finalizando 192

Capítulo 1

Computação Distribuída utilizando o Formato *JavaScript Object Notation* (JSON)

1.1 Introdução

Nesse capítulo, iremos estudar o desenvolvimento de software utilizando utilizando **arquitetura Computação Distribuída** utilizando o formato de representação de estruturas denominado de *JavaScript Object Notation* (JSON).

Neste tipo de arquitetura temos componentes denominados de **servidores** que processam as requisições, controlam as regras de negócios e fazem acesso a Servidores de Banco de Dados (SGBDs) e componentes denomina-

dos de **clientes** que controlam as interface com o usuário. Esses componentes rodam de forma desacoplados um dos outros.

Iremos aprender durante este capítulo:

- O formato JSON e sua estrutura
- Serviços de Webservices
- APIs e REST
- Computação *Back-End* e *Front-End*.

1.2 Computação Distribuída

Computação distribuída é o processo de realizar uma tarefa usando vários computadores, cada um com seu próprio hardware e software. A computação distribuída é usada para uma variedade de tarefas, como programação cliente-servidor, computação em nuvem, Big data e jogos online.

- A **programação cliente-servidor** é um modelo de computação em que os recursos são compartilhados entre os clientes e os servidores. Os clientes solicitam recursos aos servidores e os servidores fornecem esses recursos aos clientes.
  - **Importante**: no modelo cliente-servidor não é caracterizado exclusivamente pelo uso de diversos computadores, mas o uso de diversos processos ora fazendo o papel de cliente e ora fazendo o papel de

servidores. Por exemplo: em um computador pode estar rodando um processo do *Enterprise Resource Planning* (ERP) com as regras de negócio e um processo do SGBD. O **processo do ERP** é considerado servidor para os computadores clientes deste sistema, mas é considerado cliente para o **processo do SGBD**.

- **Computação em nuvem**: a computação distribuída é usada para fornecer serviços de computação em nuvem. Isso inclui serviços como armazenamento de dados, computação e processamento de dados.
- **Jogos online**: a computação distribuída é usada para executar jogos online. Isso é importante para garantir que os jogos sejam executados de forma suave e que todos os jogadores tenham uma experiência consistente.
- **Big data**: a computação distribuída é usada para processar grandes quantidades de dados. Isso é importante para uma variedade de tarefas, incluindo análise preditiva, aprendizado de máquina e mineração de dados.

1.2.1 Vantagens da Computação Distribuída

Algumas vantagens:

- A computação distribuída pode permitir que tarefas sejam **executadas mais rapidamente** do que se fossem executadas em um único computador. Isso ocorre porque vários computadores podem trabalhar na tarefa ao mesmo tempo.
- A computação distribuída pode permitir que tarefas sejam executadas em computadores que estão localizados em diferentes locais. Isso pode ser útil para tarefas que precisam acessar dados que estão localizados

em diferentes locais, ou para tarefas que precisam ser executadas rapidamente, mesmo que os computadores não estejam localizados na mesma rede.

1.2.2 Desvantagens da Computação Distribuída

Algumas desvantagens:

- A computação distribuída pode ser mais **complexa de gerenciar** do que a computação em um único computador. Isso ocorre porque os administradores de sistema precisam garantir que todos os computadores estejam funcionando corretamente e que eles estejam se comunicando entre si de forma eficaz.
- A computação distribuída pode ser mais cara do que a computação em um único computador. Isso ocorre porque as organização precisam comprar e manter vários computadores.
- Como os sistemas executam de forma desacoplada, a equipe de desenvolvimento é formada por diversos profissionais tornando mais complexa a implementação do sistema.

1.3 *JavaScript Object Notation* (JSON)

O JavaScript Object Notation (JSON) é um formato leve para armazenar e transportar dados. JSON é frequentemente usado quando os dados são enviados de um servidor para uma página da web, para outro servidor, para aplicações em celulares, etc.

O JSON é autodescritivo e fácil de se entender, ler e escrever por humanos e também pode ser facilmente interpretado por computadores.

Arquivos JSON são usados para armazenar e transmitir dados entre diferentes aplicativos. Eles são usados em uma ampla variedade de aplicativos, incluindo:

- Requisições Asynchronous JavaScript and XML (AJAX)
- *Application programming interface* (API) web
- Aplicativos móveis
- Bancos de dados não relacionais
- Ferramentas de análise de dados

Os arquivos JSON podem ser abertos e manipulados em uma variedade de linguagens de programação, incluindo JavaScript, Python, Java e PHP. Também existem ferramentas disponíveis para converter arquivos JSON para outros formatos de dados, como XML, CSV e HTML.

1.3.1 Dados JSON

Os dados JSON são gravados **como pares** de *nome-valor*. Um par *nome-valor* consiste em um nome de campo (entre aspas duplas), seguido por **dois pontos**, seguido por um valor:

```
"firstName":"John"
```

1.3.2 Objetos JSON

Os objetos JSON são escritos entre **chaves**. Assim como em *JavaScript*, os objetos podem conter vários pares nome/valor:

```
{"firstName":"John", "lastName":"Doe"}
```

1.3.3 Matrizes JSON

As matrizes JSON são escritas entre colchetes. Assim como em *JavaScript*, um *array* pode conter objetos:

```
"employees":[
    {"firstName":"John", "lastName":"Doe"},
    {"firstName":"Anna", "lastName":"Smith"},
    {"firstName":"Peter", "lastName":"Jones"}
]
```

Listing 1.1: *Array* de objetos JSON.

No exemplo da listagem 1.1 acima, o objeto *funcionários* é um *array*. Ele contém três objetos. Cada objeto é um registro de uma pessoa (com um nome e um sobrenome).

1.3.4 Objetos JSON Encadeados

Os objetos JSON podem ser encadeados formando vários níveis de profundidade:

```
"superheroes": {
  "squadName": "Super hero squad",
  "active": true,
  "members": [
    {
      "name": "Molecule Man",
      "age": 29,
      "powers": ["Radiation resistance", "Turning tiny", "Radiation blast"]
    },
    {
      "name": "Madame Uppercut",
      "age": 39,
      "powers": ["Million tonne punch","Damage resistance","Superhuman reflexes"]
    },
  ]
}
```

Listing 1.2: Objetos JSON encadeados.

1.3.5 Acessando os dados de uma objeto JSON

Para acessar os dados armazenados em um objeto JSON, temos de passar o caminho que vai deste o nome da chave do nível mais elevado até o nome da chave do nível mais interno. Veja os exemplos

- Da listagem 1.1, queremos os nomes do terceiro registro, assim fazemos:
  console.log(employees[2].firstname);

- Da listagem 1.2, queremos os dados do poder *Damage resistance*. Vamos ver essa informação por meio de um exemplo em *javascript* dentro de uma página html. Para isso, usando o bloco de notas, crie um arquivo chamado de *primeiro.html* e cole o conteúdo abaixo. **Atenção para as quebras de linhas** que podem ocorrer na variável superHeroes:

```
<script>
    var superHeroes = '{' +
        '"squadName": "Super hero squad",' +
        '"active": true,' +
        '"members": [' +
            '{"name": "Molecule Man", "age": 29, ' +
            '"powers": ["Radiation resistance", "Turning tiny"]},' +
            '{"name": "Madame Uppercut", "age": 39, ' +
            '"powers": ["Million onne punch", "Damage resistance", "Superhuman reflexes"]}' +
        ']' +
        '}';

    var superHeroesJSON = JSON.parse(superHeroes);

    console.log("superHeroesJSON:" + superHeroesJSON.members[1].powers[1]);
</script>
```

Listing 1.3: Acessando JSON por meio de *javascript*.

1.4 *WebServices*

Um *webservice* é um componente de software que funciona através da internet, permitindo a comunicação entre aplicações diferentes. Ele é uma forma de compartilhar recursos e serviços entre diferentes sistemas, independentemente de sua plataforma ou linguagem de programação.

Os *webservices* são baseados em tecnologias web, como JSON, HTML, XML e HTTP. Isso os torna fáceis de usar e integrar com outras aplicações e que pode ajudar as organizações a compartilhar recursos e serviços de forma eficiente e eficaz.

Antigamente, usava-se nos *webservices* o padrão *Simple Object Access Protocol* (SOAP), por meio de XML pesado e mais rígido. Hoje, o padrão é o *Representational State Transfer* (REST), que usa JSON, mais leve e flexível.

Os *webservices* são usados em uma ampla variedade de aplicações, incluindo:

- Integração de sistemas;
- Serviços de *e-commerce*;
- Serviços de localização;
- Serviços de jogos e
- Serviços de mídia social

Aqui estão alguns dos benefícios dos *webservices*:

- **Flexibilidade**: Os *webservices* podem ser usados para comunicar entre aplicações diferentes, independentemente de sua plataforma ou linguagem de programação;
- **Escalabilidade**: Os *webservices* podem ser facilmente escaláveis para atender às necessidades de uma organização em crescimento;
- **Segurança**: Os *webservices* podem ser usados para proteger a privacidade e a segurança dos dados;
- **Confiabilidade**: Os *webservices* podem ser usados para garantir que os serviços sejam entregues de forma confiável e consistente e
- **Eficiência**: Os *webservices* podem ser usados para automatizar tarefas e processos, o que pode economizar tempo e dinheiro.

1.5 *Application Programming Interface* (API) e *Representational State Transfer* (REST)

Uma API é um conjunto de funções que foram **implementadas em um programa de computador** que são disponibilizados para que outros programas possam utilizá-las diretamente de forma simplificada; sem envolver-se em detalhes da implementação.

Representational State Transfer (REST) não é um protocolo ou padrão, mas sim um **conjunto de restrições de arquitetura**. Os desenvolvedores de API podem implementar a arquitetura REST de maneiras variadas.

Um *endpoint* é um **endereço** de rede que identifica um**recurso** ou **serviço**. Em uma API, um *endpoint* é o **endereço de rede que identifica um recurso específico** que pode ser acessado pela API.

Os *endpoints* são geralmente nomeados de forma descritiva para refletir o recurso que eles representam. Por exemplo, o *endpoint* para o recurso *getAllUsers* (nome da função que retorna os usuários do sistema) pode ser chamado de */usuario* ou /*systemusers*. Ele combina: **Verbo HTTP (GET, POST, PUT, DELETE) + Caminho**.

Quando um cliente faz uma solicitação a um *endpoint* de uma *API REST*, essa API transfere uma representação do estado do recurso ao solicitante. Essa informação é entregue via HTTP utilizando um dos vários formatos possíveis: JSON, HTML, XML, etc. O **formato JSON** é o mais usado porque, apesar de seu nome, é independente de qualquer outra linguagem e pode ser lido por máquinas e humanos.

As *APIs REST* são baseadas nos métodos HTTP padrão, como **GET**, **POST**, **PUT** e **DELETE**. Esses métodos são usados, respectivamente, para acessar (GET), criar (POST), atualizar (PUT) e excluir (DELETE) recursos.

Os recursos são objetos que são expostos pela API. Eles podem ser entidades como *clientes, produtos ou pedidos*.

Alguns exemplos de *endpoints*:

- POST /usuario: este *endpoint* é usado para criar um novo usuário.
- GET /produtos: este *endpoint* retorna uma lista de todos os produtos.
- PUT /aluno: este *endpoint* atualiza dados de um aluno.
- DELETE /paciente: este *endpoint* remove um paciente.

1.5.1 Exemplo de API REST

A seguir temos uma API implementada em *NodeJS* que compartilha o recurso cujo *endpoint* é ***api/v1/users*** independente de domínio:

```
api.get("/api/v1/users", appUsers.getAllUsers);
```

Listing 1.4: Exemplo de código para criar uma API para o *endpoint api/v1/users*.

Aqui vemos uma API que ao ser chamada por uma aplicação cliente, vai executar a função *getAllUsers* que retorna um JSON contendo todos os usuários do sistema.

Para consumir esta API considerando o domínio *example.com*, podemos desenvolver em *javascript* o seguinte código:

```
<script>
    const request = new XMLHttpRequest();
    request.open("GET", "https://example.com/api/v1/users");
    request.send();
    request.onload = () => {
        if (request.status === 200) {
            const users = JSON.parse(request.responseText);
            console.log(JSON.stringify(users));
        } else {
            console.log("Erro:", request.status);
        }
    };
</script>
```

Listing 1.5: Exemplo de requisição para consumir a API *api/v1/users*.

Este código fará uma solicitação ao *endpoint* /api/v1/users da API em *example.com*. Se a solicitação for bem-sucedida, o código imprimirá a lista de usuários na console. Se a solicitação falhar, o código imprimirá o código de erro na console.

1.5.1.1 Exercício

- Vá ao site *https://restful-api.dev/*, escolha uma API **pública** do tipo GET e utilize-a como o exemplo 1.5.

1.5.1.2 Segurança em *API Rest*

Uma API em produção fica disponível para acessada pela Internet. Assim, qualquer pessoa que tiver o endereço da API poderá solicitar seus serviços. Caso a API não tenha algum sistema de segurança, pessoas não autorizadas podem obter dados não autorizados.

Uma das maneiras de se implementar segurança com APIs é utilizar o recurso de *token JSON Web Token* (JWT). Um *token* JWT é um pequeno trecho de dados codificado com JSON que é usado para transmitir informações de segurança entre duas partes como uma forma de autenticação. Os *tokens* JWT são normalmente usados em aplicativos da web para representar a identidade de um usuário e permitir que eles acessem recursos protegidos.

Os tokens JWT são compostos por três partes:

- ***Header*** (Cabeçalho): define o tipo de token (JWT) e o algoritmo de assinatura usado.

```
{
    "alg": "HS256",
    "typ": "JWT"
}
```

- ***Payload*** (Carga): é o corpo do *token* e contém as informações de segurança. Ele contém as *claims* (informações) da entidade tratada, normalmente o usuário autenticado. As *claims* podem ser de 3 tipos:

  - *Reserved*: atributos não obrigatórios que são usados na validação do *token* pelos protocolos de segurança das APIs.
  - *Public*: atributos que usamos em nossas aplicações. Normalmente armazenamos as informações do usuário autenticado na aplicação.
  - *Private*: atributos definidos especialmente para compartilhar informações entre aplicações.

- ***Signature*** (Assinatura): é um *hash* da cabeçalho e da carga e é usada para verificar a integridade do *token*.

```
HS256SHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload) + "." + secret_key)
```

Cujo resultado seria algo assim:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Iremos usar o *token* JWT em nossa aplicação.

1.6 Computadores *Back-End* e *Front-End*

1.6.1 Computador *Back-End*

Um computador *back-end* é um computador que **executa as funções de processamento e armazenamento de um aplicativo ou sistema**.

Normalmente, é um servidor que não é diretamente acessível aos usuários.

Os computadores *back-end* geralmente **são mais poderosos do que os computadores *front-end***, pois precisam lidar com grandes quantidades de dados e processamento.

Os computadores *back-end* são usados em uma ampla variedade de aplicativos, incluindo:

- Sites e aplicativos da web;
- Bancos de dados;
- Sistemas de arquivos;
- Servidor de e-mail;
- Servidor de rede.

1.6.2 Computador *Front-End*

Um computador *front-end* é um computador que **interage diretamente com o usuário**.

Normalmente, é um **computador pessoal ou um dispositivo móvel**.

Os computadores *front-end* geralmente **são menos poderosos do que os computadores *back-end***, pois não precisam lidar com grandes quantidades de dados e processamento.

Os computadores *front-end* são usados em uma **ampla variedade de aplicativos,** incluindo:

- Navegadores da web;
- Processadores de texto;
- Planilhas;
- Editores de imagens;
- Jogos

1.7 Finalizando

Neste capítulo foram apresentados diversos conceitos referentes a Computação Distribuída. Nos próximos capítulos iremos aprender a desenvolver softwares contendo os conceitos apresentado aqui.

Capítulo 2

Implementação do Servidor *Back-End*

2.1 Introdução

Nesse capítulo, iremos estudar o desenvolvimento de software no papel de *back-end*. Como dito, *back-end* é um computador que **executa as funções de processamento e armazenamento de um aplicativo ou sistema**.

Para o desenvolvimento do sistema exemplo, iremos usar o Node.js e alguns *frameworks* que facilitam a tarefa do programador na implementação de códigos. Iremos implementar as operações de *Create*, *Read*, *Update* e *Delete* (**CRUD**) com registros no SGBD Postgres. E ao término do capítulo, teremos APIs prontas para ser usadas pela aplicação *front-end*.

2.1.1 Diretório padrão da disciplina

Nossos programas ficarão dentro do diretório padrão *dw3*. Assim, sempre que formos criar um novo sistema, iremos criá-lo dentre desse diretório.

Para criar o diretório *dw3* faça:

1. Abra um terminal em seu sistema operacional.
2. Digite os seguintes comandos:

```
mkdir dw3
cd dw3
```

3. Para verificarmos em qual diretórios estamos, digite o comando:

```
pwd
```

Caso o retorno não seja *dw3*, temos que entrar dentro dele com o comando *cd*.

2.1.2 Repositório Remoto no *Github* da disciplina

Após criado o diretório padrão da disciplina, vamos criar o**Repositório no *Github*** para armazenar os exemplos desenvolvidos durante a aula. **Porém**, antes de criar o *repositório*, o aluno deve verificar se tem *token* ativo para

usar no momento de fazer o *push*. Caso não tenho o *token* ativo, o discente deverá criá-lo.

O aluno(a) deverá criar um **repositório remoto vazio no *Github*** sem o arquivo *README.md* no *Github* e em seguida seguir as instruções para criar o **repositório local** dentro do **diretório dw3**.

Adicionalmente, antes de efetuar o primeiro *git add .* , deve criar um arquivo de nome .gitignore:

```
touch .gitignore
```

Neste arquivo serão indicados os diretórios e arquivos que não deverão ser enviados ao **repositório remoto**. Por exemplo, a cada novo servidor *Node.js*, devemos indicar dentro do arquivo *.gitignore* o caminho do diretório **node_modules**, pois é um arquivo onde ficam instaladas as bibliotecas do *Node.js*.

Feito isso, pode dar prosseguimento na criação do **repositório local**.

A **ideia** é que a cada novo exemplo de servidor que criarmos, iremos enviá-lo para o repositório remoto, fazendo com que **o aluno tenha um portfólio** para apresentar no futuro com profissional da área.

2.2 *Node.js*

Node.js, ou simplesmente *Node*, é um *framework* de desenvolvimento de software de código aberto, gratuito e de multiplataforma. **Ele é escrito em *JavaScript***.

Node.js é usado para criar aplicativos da web que são responsivos e escaláveis. É indicado para aplicativos que precisam lidar com grandes quantidades de dados ou que precisam ser executados em vários dispositivos.

Algumas características são:

- É escrito em *JavaScript*, que é uma linguagem de programação popular e bem conhecida.
- Usa o motor *JavaScript* V8 do Google Chrome, que é um dos motores *JavaScript* mais rápidos disponíveis.
- É multiplataforma, o que significa que pode ser usada para criar aplicativos que podem ser executados em Windows, Mac e Linux.
- É um *framework* de desenvolvimento de software de código aberto, gratuito e com uma grande comunidade de usuários e desenvolvedores.

2.2.1 Instalação do Node.js

Para instalarmos o Node temos opções de acordo com o Sistema Operacional (SO) que estamos utilizando. O **site oficial do Node é *https://nodejs.org***.

Para sabermos qual a versão do Node que nosso computador possui, basta digitar o comando

```
node -v
npm -v
```

A instalação do Node no Windows, Mac e Linux por meio do site ***https://nodejs.org*** é feita de acordo com as instruções na *url https://nodejs.org/pt/download/package-manager*.

A instalação no Linux **por meio do apt-get** é de acordo com as instruções da *url https://deb.nodesource.com* .

2.2.2 Criando a primeira aplicação em Node.js

Para criarmos uma aplicação usando o Node, fazemos os seguintes passos dentro do **diretório padrão da disciplina**:

1. Crie um diretório chamado de 1_primeiro e entre dentro dele:

```
mkdir 1_primeiro
cd 1_primeiro
```

2. Instale os *frameworks* digitando o comando:

```
npm i express dotenv
```

   (a) Onde*:*
   express: é um *framework* que facilita a vida do programador.
   dotenv: permite a leitura de variáveis dentro de arquivos env.

3. Crie o arquivo *.env*:

```
touch .env
```

4. Dentro do arquivo *.env* coloque as seguintes linhas:

```
APP_NAME=1_Primeiro
PORT = 40000
```

5. Crie o arquivo *app*.js

```
touch app.js
```

6. Dentro do arquivo *app.js* coloque as seguintes linhas:

```
//@ Importa as bibliotecas
const express = require("express")
require("dotenv").config();

//@ Configura o servidor
const app = express();
const port = process.env.PORT;

//@ Cria uma rota para o endereço raiz.
app.get("/", (req, res) => {
  res.send("Hello DW3!")
})

//@ Inicia o servidor
app.listen(port, () => {
  console.log("Executando a aplicação:" , process.env.APP_NAME);
  console.log("Example app listening on port:", port);
})
```

Listing 2.1: Configurando primeiro servidor Nodejs.

7. Para executar o servidor digite:

```
node app.js
```

8. Abra um navegador e digite o endereço *http://localhost:40000*.

9. Para encerrar a execução do servidor Node, no terminal, digite *ctrl+C*.

10. Envie para o **repositório remoto** a aplicação. Não se esqueça de acrescentar o diretório *node_modules* no arquivo *.gitignore*.

```
//--  arquivo .gitignore em dw3/
node_modules/

//--  Comando git para enviar os arquivos locais para o repositório remoto
git add .
git commit -m "texto"
git push -u origin main
-- vai solicitar o usuário do github
-- vai solicitar a senha que é o TOKEN do usuário
```

2.2.3 Criando a segunda aplicação em Node.js

Vamos criar uma outra aplicação usando o padrão *Model*, *View*, *Controller* (MVC), porém, como é servidor back-end, não teremos *View* e nem *Model* por não utilizarmos SGBD. Usaremos o VSCode para implementar nosso servidor.

1. Dentro do diretório padrão, crie a outra aplicação:

```
mkdir 2_segundo
cd 2_segundo
```

2. Vamos usar o *VSCode* e instalar a extensão *Rest Client* desenvolvido por Huachao Mao. Ela vai servir para fazermos testes das APIs.

3. Instale os *frameworks* digitando o comando:

```

//--  Desta fez não iremos instalar o dotenv e nem fazer uso dele
npm i express
```

4. Crie os seguintes diretórios:

```
mkdir controller
mkdir routes
mkdir tests
```

5. Dentro do diretório *controller*, crie o arquivo *ctlHello.js* com o seguinte código:

```
//--  arquivo: controller/ctlHello.js

const hello = (req, res) => (async () => {
 res.json({ status: "ok", "mensagem": "Olá segundo!" });
})();


const helloUserGet = (request, res)  => (async () =>{
    const username = request.params.nome
    res.json({ status: "ok", "nomeusuario": username });
} )();


const helloUserPost = (request, res)  => (async () =>{
    const { username } = request.body
    res.json({ status: "ok", "nomeusuario": username });
} )();

module.exports = {
    hello,
    helloUserGet,
    helloUserPost,
}
```

Listing 2.2: Exemplo do arquivo *controller* da segunda aplicação.

6. Dentro do diretório *route*s, crie o arquivo *routes.js* com o seguinte código:

```
//--  arquivo: routes/routes.js

//@ Importa as bibliotecas e arquivos
const express = require("express");
const routerApp = express.Router();
const appHello = require("../controller/ctlHello");

//@ Configura as rotas
routerApp.get("/", appHello.hello);
routerApp.get("/helloUserGet/:nome", appHello.helloUserGet);
routerApp.post("/helloUserPost", appHello.helloUserPost);

//@ Exporta a variável com as rotas
module.exports = routerApp;
```

7. Crie o arquivo *app*.js na raiz da aplicação.

```
//-- Raiz da aplicação
touch app.js
```

8. Dentro do arquivo *app.js* coloque as seguintes linhas:

```
//--  arquivo: app.js

//@ Importa as bibliotecas e arquivos
const express = require("express");
const router = require("./routes/routes");

//@ Configura o servidor
const app = express();
const port = 40000;
app.use(express.json());
app.use(router);

//@ Inicia o servidor
app.listen(port, () => {
  console.log(`App listening at port ${port}`)

//--$
})
```

9. Para executar o servidor digite:

```
  node app.js
```

10. Agora, vamos criar os teste de requisição. Vamos usar a extensão *RESTClient* por *Huachao Mao* do *VScode*. Dentro do diretório *tests*, crie o arquivo *tests.rest* com o seguinte código:

```
//-- # arquivo: tests/tests.rest

//-- # teste de API /
###
GET http://localhost:40000/ HTTP/1.1
content-type: application/json


//-- # teste de API helloUserGet
###
GET http://localhost:40000/helloUserGet/John Doe HTTP/1.1


//-- # teste de API helloUserPost
###
POST http://localhost:40000/helloUserPost HTTP/1.1
content-type: application/json

{
  "username": "John Doe"
}
```

11. Execute os testes do arquivo *tests.rest:*

```
1 - Clique em cima do verbo POST
2 - Com o botão direito selecione a opção \emph{Send Request}
```

12. Para encerrar a execução do servidor Node, no terminal, digite *ctrl+C*.

13. Envie para o **repositório remoto** a aplicação. Não se esqueça de acrescentar o diretório *node_modules* no arquivo *.gitignore*.

```
//--  arquivo .gitignore em dw3/
node_modules/

//--  Comando git para enviar os arquivos locais para o repositório remoto
git add .
git commit -m "texto"
git push -u origin main
-- vai solicitar o usuário do github
-- vai solicitar a senha que é o TOKEN do usuário
```

2.2.4 Exercícios

Crie APIs para receberem dois números. As APIs devem retornar o resultado da operação aritmética indicada pelo nome da requisição da API.

O formato da requisição será:

```
POST http://localhost:40000/somar HTTP/1.1
content-type: application/json

{
    "num1": 4,
    "num2": 2
}
```

Requisitos:

1. Criar uma aplicação em Node para realizar cálculos com as 4 operações aritméticas;
2. A aplicação terá estrutura semelhante ao exemplo da seção 2.2.3;
3. Implementar as 4 operações aritméticas dentro do arquivo *controller/calculadora.js* por meio de funções;
4. Cada função segue o padrão das funções demonstradas no algoritmo 2.2.
5. Alterar o arquivo de *routes/route.js* acrescentando as APIs do tipo *post* referente as novas funções do *controller/calculadora.js* e

6. Alterar o arquivo de *tests* acrescentando o teste da API conforme mostrado acima.

2.3 Servidor *Back-End do* Sistema Acadêmico

Vamos dar início a implementação do servidor *back-end* de nossa aplicação exemplo.

Nossa aplicação consta de um sistema acadêmico com os módulos *Aluno* e *Curso*. **Analise seu código no repositório remoto antes de continuar**.

O *script* do banco de dados do sistema está pronto e será explicado na próxima seção.

2.3.1 Banco de dados

No repositório remoto da disciplina no *Github*, há o arquivo *dw3/backend/databaseConfig*.sql cujo conteúdo é mostrado abaixo:

```
-- arquivo: raiz do projeto/databaseConfig.sql

----- Cria um banco de dados
-- create database dw3;

create table IF NOT EXISTS cursos (
    cursoid bigserial constraint pk_cursos PRIMARY KEY,
    codigo varchar(50) UNIQUE,
    descricao VARCHAR(60),
```

```
    ativo boolean,
    deleted boolean DEFAULT false
);

insert into cursos values
    (default, 'BSI', 'Bacharelado em Sistemas de Informação', true),
    (default, 'DIREITO', 'Bacharelado em Direito', true),
    (default, 'LETRAS', 'Licenciatura em Letras', true),
    (default, 'ADM', 'Bacharelado em Administração', false)
    ON CONFLICT DO NOTHING;

create table IF NOT EXISTS alunos (
    alunoid bigserial constraint pk_alunos PRIMARY KEY,
    prontuario varchar(10) UNIQUE,
    nome varchar(50),
    endereco VARCHAR(60),
    rendafamiliar numeric(8,2),
    datanascimento date,
    cursoid bigint constraint fk_aluno_curso REFERENCES cursos,
    deleted boolean DEFAULT false
);

insert into alunos values
    (default, 'pront1', 'José das Neves', 'Rua A, Votuporanga', 6891.60, '2000-01-31',
        (SELECT cursoid from CURSOS where codigo = 'BSI')),
    (default, 'pront2', 'Maria Silveira', 'Rua B, São José do Rio Preto', 7372.41, '2002-03-12',
```

```
        (SELECT cursoid from CURSOS where codigo = 'DIREITO'))
ON CONFLICT DO NOTHING;

create table IF NOT EXISTS usuarios (
    usuarioid bigserial constraint pk_usuarios PRIMARY KEY,
    username varchar(10) UNIQUE,
    password text,
    deleted boolean DEFAULT false
);

CREATE EXTENSION if NOT EXISTS pgcrypto;

insert into usuarios values
    (default, 'admin', crypt('admin', gen_salt('bf'))), -- senha criptografada com bcrypt
    (default, 'qwe', crypt('qwe', gen_salt('bf'))) -- senha criptografada com bcrypt
ON CONFLICT DO NOTHING;

-- Usado para exercícios

create table IF NOT EXISTS clientes (
    clienteid bigserial constraint pk_clientes PRIMARY KEY,
    codigo varchar(50) UNIQUE,
    nome VARCHAR(60),
    endereco VARCHAR(50),
    ativo boolean,
    deleted boolean DEFAULT false
```

```
);

insert into clientes values
    (default, 'CLI01', 'João da Silva', 'Rua A1', true),
    (default, 'CLI02', 'Marcia Almeida', 'Rua B2', true)
    ON CONFLICT DO NOTHING;

create table IF NOT EXISTS pedidos (
    pedidoid bigserial constraint pk_pedidos PRIMARY KEY,
    numero bigint UNIQUE,
    data DATE,
    valortotal numeric(9,2),
    clienteid bigint constraint fk_pedido_cliente REFERENCES clientes,
    deleted boolean DEFAULT false
);

insert into pedidos values
 (default, 234, '2020-01-31', 6891.60, (SELECT clienteid from CLIENTES where codigo = 'CLI01'))
 ON CONFLICT DO NOTHING;
```

A sintaxe do código é compatível com o SGBD *PostgreSQL*. Analise e entenda o código antes de prosseguir com a implementação do servidor *back-end* sistema exemplo.

2.3.2 Procedimentos iniciais

Para desenvolver nosso servidor *back-end*, vamos realizar os seguintes procedimentos:

1. Vamos criar o diretório chamado dw3backend:

```
mkdir dw3backend
```

2. Dentro dele vamos instalar os frameworks:

```
npm i express pg body-parser dotenv jsonwebtoken bcryptjs
```

   Não se esqueça de adicionar o ***node_modules*** no arquivo *.gitignore*.

3. Crie o arquivo *.env* com o seguinte conteúdo:

```
APP_NAME=dw3
SECRET_API=9as1%12#xz0#@
# Banco de dados
DB_NAME=dw3
DB_USER = postgres
DB_PASS=postdba
DB_HOST=172.17.0.1
```

4. Dentro do diretório dw3backend, vamos criar os diretórios:

```
mkdir apps # contém todas as nossas aplicações
mkdir database # contém a configuração do banco de dados
mkdir routes # contém as rotas da aplicação
```

5. Crie o arquivo *app.js* com o seguinte conteúdo:

```
const express = require('express');
const bodyParser = require('body-parser');
require('dotenv').config();

const app = express();
const port = 40000;

app.listen(port, () => {
    console.log(`App listening at port ${port}`);
})
```

6. Execute o servidor:

```
  node app.js
```

7. Para parar o servidor, *digite ctrl + c*.

Importante: a apostila poderá conter apenas parte dos arquivos de código. Para a **versão completa**, use os arquivos disponibilizados no ***github***.

2.3.3 Configurando o Cross-Origin Resource Sharing (CORS)

Ao desenvolver aplicações Web, é comum que o *frontend* e o *backend* sejam executados em servidores, domínios ou portas diferentes. Nessa situação, o navegador precisa determinar se uma aplicação carregada a partir de uma determinada origem possui permissão para acessar recursos disponibilizados por outra origem. Para controlar esse comportamento existe o mecanismo denominado **Cross-Origin Resource Sharing (CORS)**, ou Compartilhamento de Recursos entre Origens Diferentes. Por exemplo, considere as seguintes URLs:

```
http://localhost:3000
http://localhost:8080
```

Apesar de utilizarem o mesmo protocolo e o mesmo endereço (`localhost`), elas possuem portas diferentes. Portanto, para o navegador, representam **origens diferentes**.

O CORS é um mecanismo baseado principalmente em cabeçalhos HTTP. O servidor backend informa ao navegador quais origens estão autorizadas a acessar seus recursos. Um dos principais cabeçalhos utilizados é:

```
Access-Control-Allow-Origin
```

Por exemplo, se o backend deseja permitir requisições provenientes do frontend executado em `http://localhost:3000`, ele pode retornar:

```
Access-Control-Allow-Origin: http://localhost:3000
```

Dessa forma, o navegador identifica que aquela origem está autorizada a acessar o recurso. Também é possível

utilizar:

```
Access-Control-Allow-Origin: *
```

O caractere * indica que o recurso pode ser acessado por qualquer origem. Entretanto, essa configuração deve ser utilizada com cuidado e não funciona como substituto para uma política adequada de autenticação e autorização.

Além do `Access-Control-Allow-Origin`, o servidor pode utilizar outros cabeçalhos para especificar quais operações são permitidas. Por exemplo:

```
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
```

Esse cabeçalho informa quais métodos HTTP podem ser utilizados.

2.3.3.1 Alterando o *app.js* para configurar o CORS

Vamos alterar o arquivo *app.js* para configurar o CORS de maneira que aceitará requisições de todos os endereço. Atenção: isso pode permitir falhas de segurança.

```
//-- routes/app.js

const express = require('express');
const bodyParser = require('body-parser');
require('dotenv').config();
```

```
const app = express();
const port = 40000;

// Adicionando o CORS

app.use((req, res, next) => {
  res.header('Access-Control-Allow-Origin', '*');
  res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content-Type, Accept,
   Authorization');

  if (req.method === 'OPTIONS') {
    return res.sendStatus(204);
  }

  next();
});


app.listen(port, () => {
    console.log(`App listening at port ${port}`);
})
```

2.3.4 Configurando o arquivo de rotas

Dentro do diretório *routes*:

1. Crie o arquivo *router.js* com o seguinte conteúdo:

```
//-- routes/routes.js $`
const express = require("express");
const routerApp = express.Router();

// middleware that is specific to this router
routerApp.use((req, res, next) => {
  next();
});

routerApp.get("/", (req, res) => {
  res.send("Olá mundo!");
});

//@ Rotas de Alunos

//@ Rotas de Cursos

// Rota Login

module.exports = routerApp;
```

2. Altere o arquivo *app*.js para ficar assim:

```
//-- arquivo: app.js
const express = require('express');
const bodyParser = require('body-parser');
require('dotenv').config();

// Importa o arquivo de rotas
const router = require('./routes/router');

const app = express();
const port = 40000;

// Configuração do CORS
app.use((req, res, next) => {
  res.header('Access-Control-Allow-Origin', '*');
  res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
  res.header('Access-Control-Allow-Headers', 'Origin, X-Requested-With, Content-Type, Accept,
   Authorization');

  if (req.method === 'OPTIONS') {
    return res.sendStatus(204);
  }

  next();
});
```

```
app.use(bodyParser.urlencoded({ extended: false, }));
app.use(express.json());

//@ Utiliza o routerApp configurado em ./routes/route.js
app.use(router);

app.listen(port, () => {
  console.log(`App listening at port ${port}`)
})

//-- $
```

3. Execute o servidor e faça o teste com o *browser* na rota /:

```
# No console
node apps.js

# No browser
localhost:40000/
```

4. Pare o servidor com *ctrl+c*.

2.3.5 Configurando o banco de dados

Dentro do diretório *database*:

1. Crie o arquivo *databaseconfig*.js com o seguinte conteúdo:

```
// ./databse/databaseconfig.js

const Pool = require('pg').Pool
const pool = new Pool({
  user: process.env.DB_USER,
  host: process.env.DB_HOST,
  database: process.env.DB_NAME,
  password: process.env.DB_PASS,
  port: process.env.DB_PORT,
```

```
})

module.exports = {
    query: (text, params) => pool.query(text, params),
  };
```

2.3.6 Configurando o diretório de aplicação: *apps*

O **diretório *apps*** contém os código de nossa aplicação. Será usado o padrão *Model*, *View* e *Controller* (MVC), porém sem o *View*. *Model*, *controller* e, também, *tests* serão subdiretórios dentro do diretório referente ao módulo.

A estrutura de diretório será semelhante à utilizada por aplicações *Django* em que cada módulo de nosso sistema terá seu próprio diretório. **Este será o padrão adotado pela disciplina**.

Como nossa aplicação contém apenas 3 módulos, *alunos, cursos e login*, iremos criar estes três diretórios:

```
//-- dw3backend/apps

mkdir alunos
mkdir cursos
mkdir login
```

2.3.7 Considerações sobre a implementação dos módulos do sistema

A seguir, iremos ver os passos necessários para implementar o módulo de login. Essa sequencia de passos deverá ser feita para cada novo módulo. **Assim, irei descrever o módulo de *login* e os demais serão criados na mesma sequência**. **São sempre 6 passos** que serão descritos na próxima seção.

Outra coisa, será mostrado na apostila e no *github* todo o código pronto para depois fazermos os testes, porém na prática, o *software* deve ser testado a cada nova rotina pronta. Não faça tudo para testá-lo, teste a cada nova rotina em desenvolvimento.

2.3.7.1 As operações de CRUD

Os módulos de *aluno* e *curso* conterão, em seus respectivos arquivos de *model* e *controller* , 5 operações do tipo CRUD, as quais são:

1. *getAllXXX*
2. *getXXXByID,*
3. *insertXXX,*
4. *updateXXX,*
5. *deleteXXX*

Cada operação CRUD será implementado em forma de função. É importante que os mesmos nomes de funções utilizado no *model* seja utilizada no *controller*. Isso facilita a vida do programador.

2.3.8 Configurando o módulo de *login*

Dentro do *apps/login* faça:

2.3.8.1 Crie os diretórios:

```
//-- apps/login

mkdir controller
mkdir model
mkdir tests
```

2.3.8.2 Configurando o *model*

Dentro do diretório *apps/login/model* crie o arquivo *mdlLogin.js* com o seguinte conteúdo:

```
//-- apps/login/model/mdlLogin.js

const db = require("../../../database/databaseconfig");

const GetCredencial = async (loginPar) => {
    return (
      await db.query(
        "select username, password " +
```

```
          "from usuarios where username = $1 and deleted = false", [loginPar]
      )
    ).rows;
  };

//--$

module.exports = {
    GetCredencial,
  };
```

2.3.8.3 Configurando o *controller*

Dentro do diretório *apps/login/controller* crie o arquivo *ctlLogin.js* com o seguinte conteúdo:

```
//-- apps/login/controller/ctlLogin.js

const jwt = require("jsonwebtoken");
const bCrypt = require("bcryptjs");
const mdlLogin = require("../model/mdlLogin");

const Login = async (req, res, next) => {
  const credencial = await mdlLogin.GetCredencial(req.body.username);
  if (credencial.length == 0) {
    return res.status(200).json({ message: "Usuário não identificado!" });
```

```
  }

  if (bCrypt.compareSync(req.body.password, credencial[0].password)) {
    //auth ok
    const username = credencial[0].username;
    const token = jwt.sign({ username }, process.env.SECRET_API, {
      expiresIn: 600, // expires in 10min
    });
    return res.json({ auth: true, token: token });
  }
  res.status(200).json({ message: "Login inválido!" });
};

function AutenticaJWT(req, res, next) {
  const tokenHeader = req.headers["authorization"];
  if (!tokenHeader)
    return res
      .status(200)
      .json({ auth: false, message: "Não foi informado o token JWT" });

  const bearer = tokenHeader.split(" ");
  const token = bearer[1];

  jwt.verify(token, process.env.SECRET_API, function (err, decoded) {
    if (err)
      return res
```

```
        .status(200)
        .json({ auth: false, message: "JWT inválido ou expirado" });

    req.userId = decoded.id;
    next();
  });
}

const Logout = (req, res, next) => {
  res.json({ auth: false, token: null });
};

module.exports = {
  Login,
  Logout,
  AutenticaJWT,
};
```

2.3.8.4 Configurando o *test*

Dentro do diretório *apps/login/tests* crie o arquivo *testLogin.rest* com o seguinte conteúdo:

```
//-- apps/login/tests/testLogin.rest

//-- # teste de login
```

```
POST http://localhost:40000/Login HTTP/1.1
content-type: application/json

{
    "username": "qwe",
    "password": "qwe"
}



//-- # teste de logout
###
POST http://localhost:40000/Logout HTTP/1.1
content-type: application/json

{

}
```

2.3.8.5 Configure as rotas

Abra o arquivo *routes/router.js* e o altere para ficar assim:

```
//-- routes/router.js
```

```
const express = require("express");
const routerApp = express.Router();

const appLogin = require("../apps/login/controller/ctlLogin");

// middleware that is specific to this router
routerApp.use((req, res, next) => {
  next();
});

routerApp.get("/", (req, res) => {
  res.send("Olá mundo!");
});

//Rotas de Alunos

//Rotas de Cursos

// Rota Login
routerApp.post("/Login", appLogin.Login);
routerApp.post("/Logout", appLogin.Logout);

module.exports = routerApp;
```

2.3.8.6 Execute os testes

Após ter implementado os códigos, execute o servidor e faça os testes:

```
# No console para iniciar o servidor
node app.js

# No VSCode para fazer os testes
No arquivo de testes do módulo login, clique com direito sobre um teste e escolha a opção Send Request.
   Analise a resposta.
```

Você deve testar cada uma das rotinas até verificar se está livre de erros. Quanto mais testes, menos chances de erros passarem. **Lembre-se**: arrumar *bug* enquanto o *software* está em desenvolvimento é MENOS custoso do que arrumar *bug* após o *software* entrar em produção.

2.3.9 Configurando o módulo de *aluno*

O módulo de *aluno* está completo no repositório do *github*. Para seu desenvolvimento, é necessário implementar os 6 passos descritos no módulo de *login*. Fique atendo para o nome dos diretórios e arquivos.

2.3.9.1 Crie os diretórios

Dentro do diretório *apps/alunos*, você deve criar os 3 diretórios *controller, model* e *tests*.

2.3.9.2 Configurando o model

Você deve criar o arquivo *apps/alunos/**model/mdl**Alunos.js* cujo conteúdo está no *github*. Abaixo, segue o código:

```
//-- arquivo apps/alunos/model/mdlAlunos.js

const db = require("../../../database/databaseconfig");

const GetAllAlunos = async () => {
  return (
    await db.query(
      "SELECT *,(SELECT descricao from CURSOS where cursoid = alunos.cursoid)" +
        "FROM alunos where deleted = false ORDER BY nome ASC"
    )
  ).rows;
};

const GetAlunoByID = async (alunoIDPar) => {
  return (
    await db.query(
      "SELECT *, (SELECT descricao from CURSOS where cursoid = alunos.cursoid)" +
        "FROM alunos WHERE alunoid = $1 and deleted = false ORDER BY nome ASC",
      [alunoIDPar]
    )
  ).rows;
};

const InsertAluno = async (alunoREGPar) => {
  //@ Atenção: aqui já começamos a utilizar a variável msg para retornor erros de banco de dados.
```

```
  let linhasAfetadas;
  let msg = "ok";
  try {
    linhasAfetadas = (
      await db.query(
        "INSERT INTO alunos " + "values(default, $1, $2, $3, $4, $5, $6, $7)",
        [
          alunoREGPar.prontuario,
          alunoREGPar.nome,
          alunoREGPar.endereco,
          alunoREGPar.rendafamiliar,
          alunoREGPar.datanascimento,
          alunoREGPar.cursoid,
          alunoREGPar.deleted,
        ]
      )
    ).rowCount;
  } catch (error) {
    msg = "[mdlAlunos|insertAlunos] " + error.detail;
    linhasAfetadas = -1;
  }

  return { msg, linhasAfetadas };
};

const UpdateAluno = async (alunoIDPar, alunoREGPar) => {
```

```
let linhasAfetadas;
let msg = "ok";
try {
  linhasAfetadas = (
    await db.query(
      "UPDATE alunos SET " +
        "prontuario = $2, " +
        "nome = $3, " +
        "endereco = $4, " +
        "rendafamiliar = $5, " +
        "datanascimento = $6, " +
        "cursoid = $7, " +
        "deleted = $8 " +
        "WHERE alunoid = $1",
      [
         alunoIDPar,
        alunoREGPar.prontuario,
        alunoREGPar.nome,
        alunoREGPar.endereco,
        alunoREGPar.rendafamiliar,
        alunoREGPar.datanascimento,
        alunoREGPar.cursoid,
        alunoREGPar.deleted,
      ]
    )
  ).rowCount;
```

```
  } catch (error) {
    msg = "[mdlAlunos|insertAlunos] " + error.detail;
    linhasAfetadas = -1;
  }

  return { msg, linhasAfetadas };
};

const DeleteAluno = async (alunoIDPar) => {
  let linhasAfetadas;
  let msg = "ok";

  try {
    linhasAfetadas = (
    await db.query(
      "UPDATE alunos SET " + "deleted = true " + "WHERE alunoid = $1",
      [alunoIDPar]
    )
  ).rowCount;
} catch (error) {
  msg = "[mdlAlunos|insertAlunos] " + error.detail;
  linhasAfetadas = -1;
}

return { msg, linhasAfetadas };
};
```

```
module.exports = {
  GetAllAlunos,
  GetAlunoByID,
  InsertAluno,
  UpdateAluno,
  DeleteAluno,
};
```

2.3.9.3 Configurando o controller

Você deve criar o arquivo *apps/alunos/**controller/ctl**Alunos.js* cujo conteúdo está no *github*.

Abaixo, segue o código:

```
//-- arquivo apps/alunos/controller/ctlAlunos.js

const mdlAlunos = require("../model/mdlAlunos");

const GetAllAlunos = (req, res) =>
  (async () => {
    let registro = await mdlAlunos.GetAllAlunos();
    for (let i = 0; i < registro.length; i++) {
      const row = registro[i];
      // Elimina parte final de uma variável data com essa: 2000-01-23T01:23:45.123
      const formattedDate = row.datanascimento.toISOString().split('T')[0];
      row.datanascimento = formattedDate;

    }
    res.json({ status: "ok", "registro": registro });
  })();

const GetAlunoByID = (req, res) =>
  (async () => {
    const alunoID = parseInt(req.params.alunoid);
    let registro = await mdlAlunos.GetAlunoByID(alunoID);



    res.json({ status: "ok", "registro": registro });
  })();
```

```
const InsertAluno = (request, res) =>
  (async () => {
    //@ Atenção: aqui já começamos a utilizar a variável msg para retornar erros de banco de dados.
    const alunoREG = request.body;
    let { msg, linhasAfetadas } = await mdlAlunos.InsertAluno(alunoREG);
    res.json({ "status": msg, "linhasAfetadas": linhasAfetadas });
  })();

const UpdateAluno = (request, res) =>
  (async () => {
    const alunoID = parseInt(request.params.alunoid);
    const alunoREG = request.body;
    let { msg, linhasAfetadas } = await mdlAlunos.UpdateAluno(alunoID, alunoREG);
    res.json({ "status": msg, "linhasAfetadas": linhasAfetadas });
  })();

const DeleteAluno = (request, res) =>
  (async () => {
    const alunoID = parseInt(request.params.alunoid);
    let { msg, linhasAfetadas } = await mdlAlunos.DeleteAluno(alunoID);
    res.json({ "status": msg, "linhasAfetadas": linhasAfetadas });
  })();

module.exports = {
  GetAllAlunos,
```

```
  GetAlunoByID,
  InsertAluno,
  UpdateAluno,
  DeleteAluno
};
```

2.3.9.4 Configurando o test

Você deve criar o arquivo *apps/alunos/**tests/test**Alunos.rest* cujo conteúdo está no *github*.

Abaixo, segue o código:

```
### Necessário para pegar o token
### Post
# @name loginAutenticado
POST http://localhost:40000/Login HTTP/1.1
content-type: application/json

{
    "username": "qwe",
    "password": "qwe"
}
### Seta o token
@accessToken = {{loginAutenticado.response.body.$.token}}
@username = {{loginAutenticado.response.body.$.username}}



### teste de inserção de aluno
POST http://localhost:40000/insertAluno HTTP/1.1
content-type: application/json
Authorization: Bearer {{accessToken}}

{
    "alunoid": 0,
    "prontuario": "pront6",
```

```
    "nome": "Carlos das Matias",
    "endereco": " Rua das couves",
    "rendafamiliar": 10000,
    "datanascimento": "1999-12-12",
    "cursoid": 2,
    "deleted": false
}

### teste de buscar todos os alunos
GET http://localhost:40000/getAllAlunos HTTP/1.1
content-type: application/json
Authorization: Bearer {{accessToken}}

{

}

### teste de buscar aluno por ID
GET http://localhost:40000/getAlunoByID/1 HTTP/1.1
content-type: application/json
Authorization: Bearer {{accessToken}}

### teste de atualização de aluno
PUT http://localhost:40000/updateAluno/1 HTTP/1.1
content-type: application/json
Authorization: Bearer {{accessToken}}
```

```
{
    "prontuario": "pront109",
    "nome": "xJosé das Neves",
    "endereco": "x Rua das couves",
    "rendafamiliar": 0,
    "datanascimento": "1999-12-12",
    "cursoid": 2,
    "deleted": false
}

### teste de atualização de aluno
DELETE http://localhost:40000/deleteAluno/1 HTTP/1.1
content-type: application/json
Authorization: Bearer {{accessToken}}
```

2.3.9.5 Configurando as rotas

Nesta etapa o arquivo *routes/router.js* deverá ficar assim:

```
//-- Arquivo routes/router.js

const express = require("express");
const routerApp = express.Router();
```

```
const appAlunos = require("../apps/alunos/controller/ctlAlunos");
const appCursos = require("../apps/cursos/controller/ctlCursos");
const appLogin = require("../apps/login/controller/ctlLogin");

// middleware that is specific to this router
routerApp.use((req, res, next) => {
  next();
});

routerApp.get("/", (req, res) => {
  res.send("Olá mundo!");
});

//Rotas de Alunos

routerApp.get("/getAllAlunos", appAlunos.GetAllAlunos);
routerApp.get("/getAlunoByID/:alunoid", appLogin.AutenticaJWT, appAlunos.GetAlunoByID);
routerApp.post("/insertAluno", appLogin.AutenticaJWT, appAlunos.InsertAluno);
routerApp.put("/updateAluno/:alunoid", appLogin.AutenticaJWT, appAlunos.UpdateAluno);
routerApp.delete("/deleteAluno/:alunoid", appLogin.AutenticaJWT, appAlunos.DeleteAluno);



//Rotas de Cursos

// Rota Login
```

```
routerApp.post("/Login", appLogin.Login);
routerApp.post("/Logout", appLogin.Logout);

module.exports = routerApp;
```

2.3.9.6 Execute os testes

Execute os testes conforme orientação feita no módulo de *login*.

2.3.10 Configurando o módulo de curso

O módulo de *curso* está completo no repositório do *github*. Para seu desenvolvimento, é necessário implementar os 6 passos descritos no módulo de *login*. Fique atendo para o nome dos diretórios e arquivos.

2.3.10.1 Crie os diretórios

Dentro do diretório *apps/cursos*, você deve criar os 3 diretórios como criado no módulo de login.

2.3.10.2 Configurando o model

Você deve criar o arquivo *apps/cursos/**model/mdl**Cursos.js* cujo conteúdo está no *github*.

2.3.10.3 Configurando o controller

Você deve criar o arquivo *apps/cursos/**controller/ctl**Cursos.js* cujo conteúdo está no *github*.

2.3.10.4 Configurando o test

Você deve criar o arquivo *apps/cursos/**tests/test**Cursos.rest* cujo conteúdo está no *github*.

2.3.10.5 Configurando as rotas

Nesta etapa o arquivo *routes/router.js* deverá ficar assim:

```
// arquivo routes/router.js
const express = require("express");
const routerApp = express.Router();

const appAlunos = require("../apps/alunos/controller/ctlAlunos");
const appCursos = require("../apps/cursos/controller/ctlCursos");
const appLogin = require("../apps/login/controller/ctlLogin");

// middleware that is specific to this router
routerApp.use((req, res, next) => {
  next();
});

routerApp.get("/", (req, res) => {
```

```
  res.send("Olá mundo!");
});

//Rotas de Alunos
routerApp.get("/getAllAlunos", appAlunos.getAllAlunos);
routerApp.get("/getAlunoByID/:alunoid", appLogin.AutenticaJWT, appAlunos.getAlunoByID);
routerApp.post("/insertAluno", appLogin.AutenticaJWT, appAlunos.insertAluno);
routerApp.update("/updateAluno/:alunoid", appAlunos.updateAluno);
routerApp.delete("/deleteAluno/:alunoid", appAlunos.deleteAluno);

//Rotas de Cursos
routerApp.get("/getAllCursos", appCursos.getAllCursos);
routerApp.get("/getCursoByID/:cursoid", appCursos.getCursoByID);
routerApp.post("/insertCurso", appCursos.InsertCurso);
routerApp.update("/updateCurso/:cursoid", appCursos.updateCurso);
routerApp.delete("/deleteCurso/:cursoid", appCursos.deleteCurso);

// Rota Login
routerApp.post("/Login", appLogin.Login);
routerApp.post("/Logout", appLogin.Logout);

module.exports = routerApp;
```

2.3.10.6 Execute os testes

Execute os testes conforme orientação feita no módulo de *login.*

2.4 Exercícios

No arquivo de *script* banco de dados desta disciplina, há a criação de duas tabelas clientes e pedidos. Implemente um servidor *backend* para elas.

2.5 Finalizando

Vimos como criar um servidor para rodar como *backend*. No próximo capítulo, iremos trabalhar com servidor *frontend*.

Capítulo 3

Implementação do Servidor *Front-End*

3.1 Introdução

Nesse capítulo, iremos estudar o desenvolvimento de software no papel de *front-end*. Como dito, *front-end* é um computador que **interage diretamente com o usuário**. Normalmente, é um **computador pessoal ou um dispositivo móvel**.

Para o desevolvimento do sistema exemplo, iremos usar o Node.js e alguns *frameworks* que facilitam a tarefa do programador na implementação de códigos no *front-end*. Iremos **implementar os formulários das operações** de *Create*, *Read*, *Update* e *Delete* (**CRUD**). E ao término do capítulo, a interface do sistema executando e interagindo-se com o servidor *back-end*.

3.2 Configuração inicial

Vamos, agora, fazer a configuração inicial de nosso servidor *front-end*. Iremos usar uma estrutura de layout pronta. A estrutura está no arquivo *frontendLayout.zip* no diretório do repositório. Após a configuração inicial do projeto, iremos iremos descompactar o dentro do diretório do frontend.

3.2.1 Instalando as bibliotecas necessárias

Vamos criar nossa aplicação *front-end* cujo diretório será *dw3frontend.* Para isso, dentro do diretório *dw3* digite o comando:

```
//-- diretório: dw3
mkdir dw3frontend
cd dw3frontend
npm i express nunjucks
```

onde *nunjucks* é um gerenciador de *template* cuja sintaxe é baseada em *jinja* do python.

Terminada a instalação, vamos criar o arquivo *app.js* e os diretórios a seguir:

```
//-- Dentro do dw3frontend
mkdir apps/
mkdir public/
mkdir routes/
mkdir views/
touch app.js
```

3.2.2 Instalando mais bibliotecas

Após analisar o conteúdo de cada diretório, vamos instalar as bibliotecas restantes:

```
npm i express-session dotenv body-parser moment http-errors
```

3.2.3 Descompactando o layout

Após instalar as bibliotecas, vamos descompactar o arquivo *frontendlayout.zip* dentro do diretório *frontend*:

```
//-- Diretório: dw3frontend

unzip frontendlayout.zip
```

3.2.4 Arquivo *env* para configuração do servidor

Vamos criar um arquivo *dw3frontend.env* para configurar nosso servidor:

1. Crie um arquivo *frontend.env* no mesmo nível do arquivo *app.js* com o seguinte conteúdo:

```
## arquivo: dw3frontend.env
APP_NAME=dw3FrontEnd
```

```
PORT=40100

## A variável servidor_dw3 guarda o endereço do servidor back-end.
SERVIDOR_DW3="http://localhost:40000"
```

3.2.5 Arquivo App.js inicial

Após instalar as bibliotecas, deixe o arquivo *app.js* como apresentado a seguir:

```
//-- app.js
//-- Para rodar: node app.js

var createError = require('http-errors');
var express = require('express');
var path = require('path');
var nunjucks = require('nunjucks');

require('dotenv').config({ path: path.join(__dirname, 'dw3frontend.env'), quiet: true });



var app = express();
var viewsPath = path.join(__dirname, 'views');
var appsPath = path.join(__dirname, 'apps');
const port = process.env.PORT || 40100;
app.set('views', [viewsPath, appsPath]);
app.set('view engine', 'njk');

nunjucks.configure(viewsPath, {
  autoescape: true,
  express: app,
  noCache: app.get('env') === 'development'
```

```
});

app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(express.static(path.join(__dirname, 'public')));

app.get('/', (req, res) => {
  res.send('Hello World!');
});

app.use(function(req, res, next) {
  next(createError(404));
});

app.use(function(err, req, res, next) {
  res.locals.message = err.message;
  res.locals.error = req.app.get('env') === 'development' ? err : {};

  res.status(err.status || 500);
  res.render('error', { title: 'Erro' });
});

app.listen(port, () => {
  console.log(`App listening at port ${port}`)
})
```

3.2.6 Executando o servidor *front-end*

Para rodar nosso servidor, faça:

1. Execute o comando:

```
node app.js
```

3.2.7 diretório *views*

Dentro do diretório *raiz-do-projeto/views*, analise cada arquivo. Aqui na apostila, iremos analisar os arquivos *layout* e *home*. Apesar da extensão *.njk* (*framework nunjucks*), vamos considerá-los como se fossem arquivos do tipo HTML.

```
<!-- views/layout.njk -->
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{{ title or 'DW3 Frontend' }}</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css">
  <link rel="stylesheet" href="/stylesheets/style.css">
</head>
<body class="app-shell">
  {% if showNavbar %}
    <nav class="navbar navbar-expand-md navbar-dark bg-dark app-navbar shadow-sm">
      <div class="container">
        <a class="navbar-brand fw-semibold" href="/home">DW3 Frontend</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
   data-bs-target="#mainNavbar" aria-controls="mainNavbar" aria-expanded="false" aria-label="Alternar
   navegação">
          <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="mainNavbar">
```

```
          <ul class="navbar-nav ms-auto mb-2 mb-md-0 gap-md-2">
            <li class="nav-item">
              <a class="nav-link {% if activeMenu == 'alunos' %}active{% endif %}"
   href="/alunos">Alunos</a>
            </li>
            <li class="nav-item">
              <a class="nav-link {% if activeMenu == 'cursos' %}active{% endif %}"
   href="/cursos">Cursos</a>
            </li>
            <li class="nav-item">
              <a class="nav-link" href="/sair">Sair</a>
            </li>
          </ul>
        </div>
      </div>
    </nav>
  {% endif %}

  <main class="app-main">
    <div class="container py-4 py-lg-5">
      {% block content %}{% endblock %}
    </div>
  </main>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
```

```
</html>
```

Arquivo *views/home.njk*. Note a importação do arquivo *layout* por meio da linha *extends "layout.njk"*.

```
<!--views/home.njk -->

{% extends "layout.njk" %}

{% block content %}
  <header class="page-header">
    <p class="page-kicker text-primary fw-semibold mb-2">Sistema administrativo</p>
    <h1 class="display-6 fw-bold mb-2">Painel principal</h1>
    <p class="text-secondary mb-0">Use a navegação superior para acessar alunos, cursos ou sair da
   aplicação.</p>
  </header>

  <div class="hero-card card border-0 shadow-sm">
    <div class="card-body p-4 p-lg-5">
      <div class="row g-4 align-items-center">
        <div class="col-lg-8">
          <h2 class="h4 fw-semibold mb-3">Bem-vindo ao sistema</h2>
          <p class="text-secondary mb-0">Esta área inicial. -static do Bootstrap.</p>
        </div>
        <div class="col-lg-4 text-lg-end">
          <a class="btn btn-primary" href="/alunos">Ir para alunos</a>
        </div>
      </div>
    </div>
```

```
  </div>
{% endblock %}
```

3.3 Implementando as rotas *index* e a *Login*

Inicialmente, vamos implementar as rotas *index* e *login*.

Para isso, vamos criar e programar o conteúdo do arquivo *routes/rtIndex.js* e *routes/rtLogin.js* da seguinte forma:

1. Criar o arquivo route/rtIndex.js

```
//-- Criar o arquivo routes/rtIndex.js
touch routes/rtIndex.js

// Conteúdo do arquivo routes/rtIndex.js

var express = require('express');
var router = express.Router();

router.get('/', function(req, res) {
  res.redirect('/login');
});

module.exports = router;
```

2. Criar o arquivo route/rtLogin.js

```
//-- Criar o arquivo routes/rtLogin.js
touch routes/rtLogin.js

// Conteúdo do arquivo routes/rtLogin.js

var express = require('express');
var router = express.Router();

router.get('/', function(req, res) {
  res.render('login', {
    title: 'Login',
    showNavbar: false
  });
});

router.post('/', function(req, res) {
  res.redirect('/home');
});

module.exports = router;
```

3. Criar o arquivo route/rtHome.js

```
    //-- Criar o arquivo routes/rtHome.js
    touch routes/rtHome.js

    // Conteúdo do arquivo routes/rtHome.js

    var express = require('express');
    var router = express.Router();

    router.get('/', function(req, res) {
      res.render('home', {
      title: 'Painel Principal',
      showNavbar: true,
      activeMenu: 'home'
    });
  });

  module.exports = router;
```

3.4 Utilizando os *layouts* e a tela de *Login*

Agora, vamos utilizar os arquivos prontos disponíveis no diretório *./views*. De início, iremos utlizar o arquivo *login.njk* e *home.njk*. Feita a consideração, vamos alterar o *app.js* para fazer uso destes dois arquivos html.

```
//-- app.js
//-- Para rodar: node app.js

var createError = require('http-errors');
var express = require('express');
var path = require('path');
var nunjucks = require('nunjucks');

// Linhas adiconadas
var indexRouter = require('./routes/rtIndex');
var loginRouter = require('./routes/rtLogin');
var homeRouter = require('./routes/rtHome');

require('dotenv').config({ path: path.join(__dirname, 'dw3frontend.env'), quiet: true });

var app = express();
var viewsPath = path.join(__dirname, 'views');
var appsPath = path.join(__dirname, 'apps');
const port = process.env.PORT || 40100;
app.set('views', [viewsPath, appsPath]);
app.set('view engine', 'njk');
```

```
nunjucks.configure(viewsPath, {
  autoescape: true,
  express: app,
  noCache: app.get('env') === 'development'
});

app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(express.static(path.join(__dirname, 'public')));

// Linhas adicionadas
app.use('/', indexRouter);
app.use('/login', loginRouter);
app.use('/home', homeRouter);

app.use(function(req, res, next) {
  next(createError(404));
});

app.use(function(err, req, res, next) {
  res.locals.message = err.message;
  res.locals.error = req.app.get('env') === 'development' ? err : {};

  res.status(err.status || 500);
  res.render('error', { title: 'Erro' });
```

```
});

app.listen(port, () => {
  console.log(`App listening at port ${port}`)
})
```

3.5 Implementando a lógica do *Login*

Crie os diretórios:

```
mkdir apps/login
cd apps/login
```

Em seguida, vamos criar o arquivo *vwLogin.js* em *apps/login*:

```
//Arquivo apps/login/vwLogin.js

document.addEventListener('DOMContentLoaded', function() {
  var form = document.getElementById('loginForm');
  var errorBox = document.getElementById('loginError');

  if (!form) {
    return;
  }

  form.addEventListener('submit', async function(event) {
    event.preventDefault();

    var servidorDw3 = form.dataset.servidorDw3; // Valor que vem de routes/rtLogin.js
    var usuario = document.getElementById('usuario').value;
    var senha = document.getElementById('senha').value;
    var submitButton = form.querySelector('button[type="submit"]');

    if (errorBox) {
      errorBox.classList.add('d-none');
      errorBox.textContent = '';
    }

    if (submitButton) {
```

```
    submitButton.disabled = true;
    submitButton.textContent = 'Entrando...';
  }

  try {
    var response = await fetch(servidorDw3 + '/Login', {
      method: 'POST',
      headers: {
        'content-type': 'application/json'
      },
      body: JSON.stringify({
        UserName: usuario,
        Password: senha
      })
    });

    if (!response.ok) {
      throw new Error('Usuario ou senha inválidos.');
    }

    var data = await response.json();
    var token = data.token || data.Token || data.accessToken || data.AccessToken;

    if (!token) {
      throw new Error('Erro no login. Token não retornado pelo servidor.');
    }
```

```
      localStorage.setItem('token', token); // Salva o token para ser usado na chamadas ao backend.
      document.cookie = 'IsLogged=true; path=/'; // Usado ara verificar se o usuário está ou não logado.
      window.location.href = '/home';

    } catch (error) {
      if (errorBox) {
        errorBox.textContent = error.message || 'Nao foi possivel realizar o login.';
        errorBox.classList.remove('d-none');
      }
    } finally {
      if (submitButton) {
        submitButton.disabled = false;
        submitButton.textContent = 'Entrar';
      }
    }
  });
});
```

3.5.1 Criando a *View* de *Login*

No diretório *views*, temos o arquivo *login.njk* cujo conteúdo é:

```
// arquivo views/login.njk

{% extends "layout.njk" %}

{% block content %}
  <section class="login-screen d-flex align-items-center justify-content-center">
    <div class="login-card card shadow-lg border-0">
      <div class="card-body p-4 p-md-5">
        <p class="page-kicker text-primary fw-semibold mb-2">DW3 Frontend</p>
        <h1 class="h3 fw-bold mb-2">Acesso ao sistema</h1>
        <p class="text-secondary mb-4">Entre para visualizar o painel e o menu principal.</p>

        <form id="loginForm" data-servidor-dw3="{{ servidorDw3 }}">
          <div id="loginError" class="alert alert-danger d-none" role="alert"></div>
          <div class="mb-3">
            <label class="form-label" for="usuario">Usuário</label>
            <input class="form-control" id="usuario" name="usuario" type="text" placeholder="Seu usuário"
   autocomplete="username">
          </div>
          <div class="mb-4">
            <label class="form-label" for="senha">Senha</label>
            <input class="form-control" id="senha" name="senha" type="password" placeholder="Sua senha"
```

```
   autocomplete="current-password">
          </div>
          <button class="btn btn-primary w-100" type="submit">Entrar</button>
        </form>
      </div>
    </div>
  </section>
  <script src="/apps/login/vwLogin.js"></script>  <!-- NOTE ue aqui é importado a lógica do Login -->
{% endblock %}
```

3.5.2 Alterando a rota do *Login*

Altere o conteúdo do arquivo *./routes/rtLogin.js* para acrescer uma variável exportada para o arquivo *./views/login.njk* que contém o endereço que roda o servidor *backend*:

```
// arquivo: ./routes/rtLogin.js

var express = require('express');
var router = express.Router();

router.get('/', function(req, res) {
  res.render('login', {
    title: 'Login',
    showNavbar: false,
```

```
    servidorDw3: process.env.SERVIDOR_DW3 // Linha adicionada
  });
});

router.post('/', function(req, res) {
  res.redirect('/home');
});

module.exports = router;
```

Pronto, já podemos rodar o teste de login. Para isso digite na raiz-do-projeto

```
  node app.js

  Use a credencial:
  user: qwe
  senha: qwe
```

3.5.3 Acrescentando verificação de usuário autenticado

Uma vez que o *login* esteja funcionando corretamente, é possível notar uma variável do tipo *Cookie* chamada *IsLogged*. Essa variável possibilita saber se o acesso à uma página do sistema é feito por um usuário autenticado ou não. Desta forma, vamos criar uma rotina que verifica se o usuário está autenticado toda vez que acessar uma página do sistema e, caso não esteja, ele será redirecionado para a tela de *login*.

Essa função será chamada toda a vez que uma página html for carregada. Para isso, faça:

1. Crie o arquivo *dw3frontend.js* no diretório *./public/js* que também deverá ser criado:

```
// Criar o diretório ./public/js
mkdir ./public/js

// Criar o arquivo dw3frontend dentro do diretório criado.
touch dw3frontend.js
```

2. Implemente a rotina *dw3IsLogged()* de verificação de usuário autenticado no arquivo *./public/js/dw3frontend.js*.

```
// Arquivo:  ./public/js/dw3frontend.js


function dw3IsLogged() {
  // Verifica se o Cookie IsLogged existe.
  const isLogged = document.cookie
    .split(';')
    .some((cookie) => cookie.trim().startsWith('IsLogged='));

  if (!isLogged) {
    window.location.href = '/login';
    return false;
  }
```

```
    return true;
}
```

3.6 Implementando a lógica de *Home*

Os arquivos do módulo *home* serão *./apps/home/vwHome.js* e *./views/home.njk*.

Vamos criar o diretório *./apps/home* e dentro dele o arquivo *vwHome.js*.

```
# Criar o  diretório ./apps/home
mkdir ./app/home

# Criar o arquivo vwHome.js
touch vwHome.js
```

No *vwHome.js* verificar se o usuário está logado por meio do uso da função *dw3frontend*. Logo que a página é aberta, devemos verificar se o usuário está autenticado ou não usando a função *document.addEventListener()*.

Vamos implementar o arquivo *vsHome.js*:

```
# Arquivo: ./apps/home/vwHome.js
```

```
  document.addEventListener('DOMContentLoaded', function() {
    dw3IsLogged(); // Função em ./public/js/dw3frontend.js que verifica se o cookie IsLogged existe ou
   não.
  });
```

Para finalizar o */home*, vamos importar os arquivo *dw3frontend.js* e *vwHomme.js* no final do arquivo *./views/home.njk*.

```
  # Arquivo: ./views/home.njk

  {% extends "layout.njk" %}

  {% block content %}
  <header class="page-header">
  ...
  </header>

  <div class="hero-card card border-0 shadow-sm">
  ...
  </div>

  <script src="/js/dw3frontend.js"></script> // Linha inserida
  <script src="/apps/home/vwHome.js"></script> // Linha inserida
{% endblock %}
```

3.7 Implementando a ação de *logout* do sistema

Para finalizarmos o processo de autenticação de usuários, vamos implementar a rotina de sair (*logout*) do sistema. Para isso, precisamos implementar no arquivo *./public/js/dw3frontend.js* a rotina *dw3ClearIsLoggedCookie*, importar o *dw3frontend.js* no arquivo *./views/login.njk*, inserir, logo no começo do arquivo *./apps/login/vwLogin.js*, função *dw3ClearIsLoggedCookie()* e redirecionar o link do menu Sair do arquivo *./views/layout.njk* para a rota */login* que o usuário não estará mais autenticado.

Para isso faça:

1. Crie a função *dw3ClearIsLoggedCookie* no arquivo *./public/js/dw3frontend.js*:

```
# Arquivo: ./public/js/dw3frontend.js

function dw3ClearIsLoggedCookie() {
  // Limpa o Cookie Islogged
  var isLogged = document.cookie
    .split(';')
    .some((cookie) => cookie.trim().startsWith('IsLogged='));

  if (isLogged) {
    document.cookie = 'IsLogged=; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT';
  }
}
```

2. Importe o arquivo *./public/js/dw3frontend.js* no formulário *./views/login.njk*:

```
# Arquivo: ./views/login.njk

# Antes
  </section>

  <script src="/apps/login/vwLogin.js"></script>
  {% endblock %}

# Depois

  </section>
  <script src="/js/dw3frontend.js"></script> // Linha adicionada
  <script src="/apps/login/vwLogin.js"></script>
{% endblock %}
```

3. Troque a *tage href* do arquivo *./views/layout.njk* de */sair* para */login*:

```
# Arquivo: ./views/layout.njk

# Troque
...
<li class="nav-item">
   <a class="nav-link" href="/sair">Sair</a>
</li>
...

# Por

<li class="nav-item">
   <a class="nav-link" href="/login">Sair</a>
</li>
```

4. Adicione a função *dw3ClearIsLoggedCookie* no início do arquivo *./apps/login/vwLogin.js*:

```
# Arquivo: ./apps/login/vwLogin.js


document.addEventListener('DOMContentLoaded', function() {
  // Limpa o Cookie Islogged
  dw3ClearIsLoggedCookie();

  var form = document.getElementById('loginForm');
  ...
```

3.8 Implementando o módulo de Alunos

Vamos implementar o frontend do módulo de alunos de nosso sistema acadmico. Para isso, iremos usar o diretório **./apps/módulo** para implementar os view referentes a um módulo específico. Não iremos usar mais o diretório *./views*, mas sim separar em cada diretório *./apps/módulo* os arquivos necessários para a execução do módulo.

A regra geral para a implementação é seguir os passos:

1. Crie o arquivo de lista em *./apps/módulo*.
2. Crie uma **rota** em *./routes/módulo* que abra o formulário daquele módulo.
3. Importe a rota no arquivo *./app.js*.

4. Faça o teste para ver se o formulário é aberto.

5. Crie o arquivo de formulário de campos em *./apps/módulo*.

3.8.1 Criando o arquivo de lista de *Alunos*

Nesse arquivo, vamos implementar o formulário que **lista todos os alunos** cadastrados no sistema. Vamos chamar esse tipo de formulário de **vwLstMódulo.njk**, no nosso caso, vamos chamar de *vwLstAluno.njk* :

No diretório *./apps*, crie o diretório *alunos* e o arquivo *vwLstAlunos.njk*:

```
## Criando o diretório de alunos
mkdir ./apps/alunos

## Criando o arquivo ./apps/alunos/vwLstAlunos.njk
touch ./Apps/alunos/vwLstAlunos.njk
```

Agora, o arquivo *vwLstAlunos.njk* conterá o código abaixo. **Tome cuidado** para não copiar a linha // Arquivo ... :

```
// Arquivo: ./apps/alunos/vwLstAlunos.njk

{% extends "layout.njk" %}

{% block content %}
  <header class="page-header">
```

```
    <p class="page-kicker text-primary fw-semibold mb-2">Cadastro</p>
    <h1 class="h3 fw-bold mb-2">Alunos</h1>
    <p class="text-secondary mb-0">Tela base para cadastro e consulta de alunos.</p>
  </header>

  <div class="card page-card border-0 shadow-sm">
    <div class="card-body p-4">
      <p class="mb-0 text-secondary">Conteúdo da área de alunos.</p>
    </div>
  </div>
{% endblock %}
```

3.8.2 Criando a rota de *Alunos*

No diretório *./routes*, crie o arquivo *rtAlunos.js* e coloque o código abaixo. Também, passamos como parâmetro a variável **process.env.SERVIDOR_DW3**, utilizada para configurar a *url* do servidor *backend*:

```
// arquivo: ./routes/rtAlunos.js
touch ./routes/rtAlunos.js


// Código
// arquivo: ./routes/rtAlunos.js

var express = require('express');
var router = express.Router();

router.get('/', function(req, res) {
  res.render('alunos/vwLstAlunos', {
    title: 'Alunos',
    showNavbar: true,
    activeMenu: 'alunos',
    servidorDw3: process.env.SERVIDOR_DW3 // Usado para configurar a url do backend.
  });
});

module.exports = router;
```

3.8.3 Alterando o arquivo *app.js* para adicionar a rota */alunos* e outras configurações

O arquivo *apps* deve ser alterado para o registro da rota */alunos* e configurado Nunjucks/Express para buscar templates em *views* e, também, em *apps*. O arquivo ficará assim:

```
// Arquivo: app.js

var createError = require('http-errors');
var express = require('express');
var path = require('path');
var nunjucks = require('nunjucks');

var indexRouter = require('./routes/rtIndex');
var loginRouter = require('./routes/rtLogin');
var homeRouter = require('./routes/rtHome');
var alunosRouter = require('./routes/rtAlunos');

require('dotenv').config({ path: path.join(__dirname, 'dw3frontend.env'), quiet: true });

var app = express();
var viewsPath = path.join(__dirname, 'views');
var appsPath = path.join(__dirname, 'apps'); // Linha adicionada
const port = process.env.PORT || 40100;
app.set('views', [viewsPath, appsPath]);
app.set('view engine', 'njk');

nunjucks.configure([viewsPath, appsPath], {
  autoescape: true,
  express: app,
  noCache: app.get('env') === 'development'
```

```
});

app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(express.static(path.join(__dirname, 'public')));
app.use('/apps', express.static(path.join(__dirname, 'apps')));

app.use('/', indexRouter);
app.use('/login', loginRouter);
app.use('/home', homeRouter);
app.use('/alunos', alunosRouter); // Linha adicionada

app.use(function(req, res, next) {
  next(createError(404));
});

app.use(function(err, req, res, next) {
  res.locals.message = err.message;
  res.locals.error = req.app.get('env') === 'development' ? err : {};

  res.status(err.status || 500);
  res.render('error', { title: 'Erro' });
});

app.listen(port, () => {
  console.log(`App listening at port ${port}`)
```

```
})

// $``
```

Agora teste a aplicação executando:

```
## Testando a aplicação

node app.js
```

3.8.4 Adicionando lógica ao formulário *vwLstAlunos*

O arquivo *vwLstAlunos* não possui nenhuma ainda. Quando o abrimos vemos apenas uma página estática. Vamos, agora, colocar lógica nele. Esse arquivo será responsável por listar todos os alunos cadastrados e dispnibilizar ações para inclusão, remoção, visualização e atualização de alunos. Visualmente, ao formulário inicial, vamos adicionar um botão para inserir um novo aluno e a tabela (`https://datatables.net/`) com as operações de visualizar, atualizar e remover um aluno.

Com isso o arquivo vwLstAlunos.njk ficará assim:

```
// arquivo: ./apps/alunos/vwLstAlunos.njk


{% extends "layout.njk" %}

{% block content %}
  <link rel="stylesheet" href="https://cdn.datatables.net/2.1.8/css/dataTables.bootstrap5.css">
  <link rel="stylesheet"
   href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">

  <header class="page-header">
    <p class="page-kicker text-primary fw-semibold mb-2">Cadastro</p>
    <h1 class="h3 fw-bold mb-2">Alunos</h1>
    <p class="text-secondary mb-0">Consulte os alunos cadastrados no sistema.</p>
  </header>

  <div class="card page-card border-0 shadow-sm">
    <div class="card-body p-4">
      <div class="d-flex flex-column flex-md-row align-items-md-center justify-content-md-between gap-3
   mb-4">
        <div>
          <h2 class="h5 fw-semibold mb-1">Lista de alunos</h2>
          <p class="text-secondary mb-0">Alunos retornados pelo servidor backend.</p>
        </div>
        <a class="btn btn-primary" href="/alunos/form?oper=Cr" id="btnNovoAluno">Incluir novo aluno</a>
```

```
//Linha adicionada
   </div>

   <div id="alunosAlert" class="alert alert-danger d-none" role="alert"></div> //Linha adicionada. Mas
será removida à frente.

   <div id="alunosListagem" data-servidor-dw3="{{ servidorDw3 }}"> // Linha adicionada.Linha
adicionada.  data-servidor-dw3 vai virar no vwLstAlunos dataset.servidorDw3 .
     <div class="table-responsive">
       <table id="tblAlunos" class="table table-striped table-hover align-middle w-100">
         <thead>
           <tr>
             <th>ID</th>
             <th>Prontuario</th>
             <th>Nome</th>
             <th>Endereco</th>
             <th>Renda Familiar</th>
             <th>Data Nascimento</th>
             <th>Curso</th>
             <th>Operacoes</th>
           </tr>
         </thead>
         <tbody></tbody>
       </table>
     </div>
   </div>
```

```
    </div>
  </div>

  <script src="/js/dw3frontend.js"></script>
  <script src="https://code.jquery.com/jquery-4.0.0.slim.min.js"></script> //Linha adicionada
  <script src="https://cdn.datatables.net/2.1.8/js/dataTables.js"></script> //Linha adicionada
  <script src="https://cdn.datatables.net/2.1.8/js/dataTables.bootstrap5.js"></script> //Linha adicionada
  <script src="/apps/alunos/vwLstAlunos.js"></script>
{% endblock %}
```

3.8.5 Criando o arquivo *vwLstAlunos*

É no arquivo *./apps/alunos/vwLstAlunos.js* que ficar a implementação do código que executa as ações do formulário *./apps/alunos/vwLstAlunos.njk*. Vamos implementá-lo:

```
// arquivo: ./apps/alunos/vwLstAlunos.js


document.addEventListener('DOMContentLoaded', function() {
  if (!dw3IsLogged()) {
    return;
  }

  var listagem = document.getElementById('alunosListagem');
  var alertBox = document.getElementById('alunosAlert');

  if (!listagem) {
    return;
  }

  var servidorDw3 = listagem.dataset.servidorDw3;

  configurarOperacoesAlunos(alertBox);
  carregarAlunos(servidorDw3, alertBox);
});



async function carregarAlunos(servidorDw3, alertBox) {
  try {
```

```
    if (!servidorDw3) {
      throw new Error('Endereco do servidor backend nao configurado.');
    }

    var response = await fetch(servidorDw3 + '/getAllAlunos');

    if (!response.ok) {
      throw new Error('Nao foi possivel carregar a lista de alunos.');
    }

    var data = await response.json();

    if (data.status !== 'ok' || !Array.isArray(data.registro)) {
      throw new Error('Resposta invalida do servidor backend.');
    }

    inicializarTabelaAlunos(data.registro);
  } catch (error) {
    exibirErro(alertBox, error.message || 'Erro ao carregar alunos.');
    inicializarTabelaAlunos([]);
  }
}



function inicializarTabelaAlunos(alunos) {
```

```
new DataTable('#tblAlunos', {
  data: alunos,
  columns: [
    { data: 'alunoid', defaultContent: '' },
    { data: 'prontuario', defaultContent: '' },
    { data: 'nome', defaultContent: '' },
    { data: 'endereco', defaultContent: '' },
    {
      data: 'rendafamiliar',
      defaultContent: '',
      render: function(data) {
        if (data === null || data === undefined || data === '') {
          return '';
        }

        return Number(data).toLocaleString('pt-BR', {
          style: 'currency',
          currency: 'BRL'
        });
      }
    },
    {
      data: 'datanascimento',
      defaultContent: '',
      render: function(data) {
        if (!data) {
```

```
          return '';
        }

        return new Date(data + 'T00:00:00').toLocaleDateString('pt-BR');
      }
    },
    { data: 'descricao', defaultContent: '' },
    {
      data: null,
      orderable: false,
      searchable: false,
      render: function(data, type, row) {
        if (type !== 'display') {
          return '';
        }

        var alunoId = escaparHtml(row.alunoid || '');

        return [
          '<div class="btn-group btn-group-sm" role="group" aria-label="Operacoes do aluno">',
          '<button class="btn btn-outline-primary" type="button" data-action="visualizar"
data-alunoid="' + alunoId + '" title="Visualizar" aria-label="Visualizar"><i class="bi
bi-eye"></i></button>',
          '<button class="btn btn-outline-secondary" type="button" data-action="atualizar"
data-alunoid="' + alunoId + '" title="Atualizar" aria-label="Atualizar"><i class="bi
bi-pencil-square"></i></button>',
```

```
            '<button class="btn btn-outline-danger" type="button" data-action="remover" data-alunoid="' +
   alunoId + '" title="Remover" aria-label="Remover"><i class="bi bi-trash"></i></button>',
            '</div>'
          ].join('');
        }
      }
    ],
    language: {
      emptyTable: 'Nenhum aluno cadastrado.',
      info: 'Mostrando _START_ ate _END_ de _TOTAL_ registros',
      infoEmpty: 'Mostrando 0 ate 0 de 0 registros',
      infoFiltered: '(filtrado de _MAX_ registros no total)',
      lengthMenu: 'Mostrar _MENU_ registros',
      loadingRecords: 'Carregando...',
      processing: 'Processando...',
      search: 'Pesquisar:',
      zeroRecords: 'Nenhum registro encontrado',
      paginate: {
        first: 'Primeiro',
        last: 'Ultimo',
        next: 'Proximo',
        previous: 'Anterior'
      }
    }
  });
}
```

```
function configurarOperacoesAlunos(alertBox) {
  var tabela = document.getElementById('tblAlunos');

  if (!tabela) {
    return;
  }

  tabela.addEventListener('click', function(event) {
    var botao = event.target.closest('button[data-action][data-alunoid]');

    if (!botao) {
      return;
    }

    var action = botao.dataset.action;
    var alunoId = botao.dataset.alunoid;

    if (action === 'visualizar') {
      exibirAviso(alertBox, 'Visualizar aluno ' + alunoId + '.');
      return;
    }

    if (action === 'atualizar') {
```

```
      exibirAviso(alertBox, 'Atualizar aluno ' + alunoId + '.');
      return;
    }

    if (action === 'remover') {
      if (confirm('Confirma a remocao do aluno ' + alunoId + '?')) {
        exibirAviso(alertBox, 'Remover aluno ' + alunoId + '.');
      }
    }
  });
}



function exibirAviso(alertBox, mensagem) {
  if (!alertBox) {
    return;
  }

  alertBox.textContent = mensagem;
  alertBox.classList.remove('alert-danger');
  alertBox.classList.add('alert-info');
  alertBox.classList.remove('d-none');
}

function escaparHtml(valor) {
```

```
  return String(valor)
    .replace(/&/g, '&amp;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;');
}

function exibirErro(alertBox, mensagem) {
  if (!alertBox) {
    return;
  }

  alertBox.textContent = mensagem;
  alertBox.classList.remove('alert-info');
  alertBox.classList.add('alert-danger');
  alertBox.classList.remove('d-none');
}
```

3.8.6 Criando as Rotas de *Alunos*

Dentro do diretório *routes/* crie o arquivo *rtAlunos.js* com o seguinte conteúdo:

```
//Arquivo routes/rtAlunos.js

var express = require('express');
var router = express.Router();

router.get('/', function(req, res) {
  res.render('alunos/vwLstAlunos', {
    title: 'Alunos',
    showNavbar: true,
    activeMenu: 'alunos',
    servidorDw3: process.env.SERVIDOR_DW3 // É necessário, pois, o front em html não acesso ao arquivo
   .env
  });
});

module.exports = router;
```

No arquivo *app.js*, logo abaixo da linha *var homeRoute = require('./routes/rtHome')*, acrescente a linha *var alunosRouter = require('./routes/rtAlunos');* e abaixo de *app.use('/home', homeRouter);*, a linha *app.use('/alunos', alunosRouter);*

O arquivo completo está abaixo:

```
// Arquivo app.js
var createError = require('http-errors');
var express = require('express');
var path = require('path');
var nunjucks = require('nunjucks');

var indexRouter = require('./routes/rtIndex');
var loginRouter = require('./routes/rtLogin');
var homeRouter = require('./routes/rtHome');
var alunosRouter = require('./routes/rtAlunos'); // Linha adicionda.

require('dotenv').config({ path: path.join(__dirname, 'dw3frontend.env'), quiet: true });

var app = express();
var viewsPath = path.join(__dirname, 'views');
var appsPath = path.join(__dirname, 'apps');
const port = process.env.PORT || 40100;
app.set('views', [viewsPath, appsPath]);
app.set('view engine', 'njk');

nunjucks.configure([viewsPath, appsPath], {
  autoescape: true,
  express: app,
  noCache: app.get('env') === 'development'
});
```

```
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(express.static(path.join(__dirname, 'public')));
app.use('/apps', express.static(path.join(__dirname, 'apps')));

app.use('/', indexRouter);
app.use('/login', loginRouter);
app.use('/home', homeRouter);
app.use('/alunos', alunosRouter); // Linha adicionada.


app.use(function(req, res, next) {
  next(createError(404));
});

app.use(function(err, req, res, next) {
  res.locals.message = err.message;
  res.locals.error = req.app.get('env') === 'development' ? err : {};

  res.status(err.status || 500);
  res.render('error', { title: 'Erro' });
});

app.listen(port, () => {
  console.log(`App listening at port ${port}`)
```

```
})

//$
```

Agora, rode o servidor.

```
// Na raiz, execute:

node app.js
```

Navegue até o módulo de Alunos.

3.8.7 Implementando o formulário de Alunos

No diretório *./apps/alunos*, vamos criar o formulário que será usado para realizamos o cadastro, visualização, atualização e remocão de alunos. O formulário irá se chamar *vwFormAlunos.njk* e nele ficarão os campos, botões e demais componentes gráficos para manipularmos os dados dos alunos.

```
<!-- Arquivo: ./apps/alunos/vwFormAlunos.njk -->


{% extends "layout.njk" %}

{% block content %}
  <link rel="stylesheet"
   href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/select2@4.1.0-rc.0/dist/css/select2.min.css">
  <link rel="stylesheet"
   href="https://cdn.jsdelivr.net/npm/select2-bootstrap-5-theme@1.3.0/dist/select2-bootstrap-5-theme.min.css">

  <header class="page-header">
    <p class="page-kicker text-primary fw-semibold mb-2">Cadastro</p>
    <h1 class="h3 fw-bold mb-2">Formulario de alunos</h1>
    <p class="text-secondary mb-0">Manipule os dados cadastrais dos alunos.</p>
  </header>

  <div class="card page-card border-0 shadow-sm">
    <div class="card-body p-4">
```

```
    <div class="d-flex flex-column flex-md-row align-items-md-center justify-content-md-between gap-3
mb-4">
      <div>
        <h2 class="h5 fw-semibold mb-1">Dados do aluno</h2>
        <p class="text-secondary mb-0">Informe os dados para inserir, atualizar ou remover um aluno.</p>
      </div>
    </div>

    <form id="frmAlunos" data-servidor-dw3="{{ servidorDw3 }}">
      <input type="hidden" id="alunoid" name="alunoid" value="0">
      <input type="hidden" id="deleted" name="deleted" value="false">

      <div class="row g-3">
        <div class="col-12 col-md-4">
          <label for="prontuario" class="form-label">Prontuario</label>
          <input type="text" class="form-control" id="prontuario" name="prontuario" value="" required>
        </div>

        <div class="col-12 col-md-8">
          <label for="nome" class="form-label">Nome</label>
          <input type="text" class="form-control" id="nome" name="nome" value="" required>
        </div>

        <div class="col-12">
          <label for="endereco" class="form-label">Endereco</label>
          <input type="text" class="form-control" id="endereco" name="endereco" value="" required>
```

```
        </div>

        <div class="col-12 col-md-4">
          <label for="rendafamiliar" class="form-label">Renda Familiar</label>
          <input type="number" class="form-control" id="rendafamiliar" name="rendafamiliar" min="0"
step="0.01" value="" required>
        </div>

        <div class="col-12 col-md-4">
          <label for="datanascimento" class="form-label">Data Nascimento</label>
          <input type="date" class="form-control" id="datanascimento" name="datanascimento" value=""
required>
        </div>

        <div class="col-12 col-md-4">
          <label for="cursoid" class="form-label">Curso</label>
          <select class="form-select" id="cursoid" name="cursoid" required>
            <option value="">Selecione um curso</option>
          </select>
        </div>
      </div>

      <div class="d-flex flex-column flex-sm-row gap-2 mt-4">
        <button type="button" class="btn btn-primary  d-none" id="btnInserirAluno">
          <i class="bi bi-plus-lg me-1"></i>Inserir
        </button>
```

```
          <button type="button" class="btn btn-warning  d-none" id="btnAtualizarAluno">
            <i class="bi bi-pencil-square me-1"></i>Atualizar
          </button>
          <button type="button" class="btn btn-danger d-none" id="btnRemoverAluno">
            <i class="bi bi-trash me-1"></i>Remover
          </button>
          <a class="btn btn-outline-secondary" href="/alunos" id="btnFecharAluno">
            <i class="bi bi-x-lg me-1"></i>Fechar
          </a>
        </div>
      </form>
    </div>
  </div>

  <script src="https://code.jquery.com/jquery-4.0.0.slim.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/select2@4.1.0-rc.0/dist/js/select2.min.js"></script>
  <script>
    $(function () {
      $('#cursoid').select2({
        theme: 'bootstrap-5',
        placeholder: 'Selecione um curso',
        width: '100%'
      });
    });
  </script>
{% endblock %}
```

3.8.8 Adicionado a rota para o formulário de Alunos

Uma vez que o formulário de Alunos está pronto, vamos criar uma rota para chegarmos nele. Veja, em *app.js* já importamos *./routes/rtAlunos.js* e criamos a rota */aluno*. Agora vamos alterar o arquivo de rotas de alunos e adicionar uma nova rota chamada de */form* que ficará subordinada a rota */alunos*. Assim, **para abrir o vwFormAlunos.njk, passaremos a rota completa */alunos/form***.

Abra o arquivo *./routes/rtAlunos.js* e adicione a nova rota:

```
// Arquivo: ./routes/rtAlunos.js


var express = require('express');
var router = express.Router();

router.get('/', function(req, res) {
  res.render('alunos/vwLstAlunos', {
    title: 'Alunos',
    showNavbar: true,
    activeMenu: 'alunos',
    servidorDw3: process.env.SERVIDOR_DW3 // É necessário, pois, o front em html não acesso ao arquivo
   .env
  });
```

```
});


// Nova rota
router.get('/form', function(req, res) {
  res.render('alunos/vwFormAlunos', {
    title: 'Formulario de alunos',
    showNavbar: true,
    activeMenu: 'alunos',
    servidorDw3: process.env.SERVIDOR_DW3
  });
});

module.exports = router;
```

Reexecute o servidor Node.js.

```
// Na raiz, execute:

node app.js
```

Agora, navegue até Inserir Alunos.

3.8.9 Implementando o cadastro de um novo Aluno

Uma vez que o formulário do módulo *Alunos* está pronto, vamos implementar a rotina cadastro. Toda a lógica das operações CRUDs ficarão no arquivo *vsFormAlunos.js*.

Antes, porém, de iniciarmos a implementação do CRUDs de Alunos, vamos criar uma API para trazer os cursos disponíveis que serão mostrados no componente *Select* do formulário de Alunos (vsFormAlunos.njk). A API será implementada no *backend*, nos arquivos de *model* e *controller*. Também iremos alterar rotas e testes para a API.

1. No arquivo de *model*

```
// Arquivo: backend/apps/alunos/model/mdlAlunos.js
...
...
const GetCursosToAlunos = async () => {
  return (await db.query("select cursoid, descricao from cursos")).rows;
};
...
...


module.exports = {
  GetAllAlunos,
  GetAlunoByID,
  GetCursosToAlunos, // Linha adicionada
  InsertAluno,
  UpdateAluno,
```

```
  DeleteAluno,
};
```

2. No arquivo de *controller*

```
// Arquivo: backend/apps/alunos/controller/ctlAlunos.js

...
...

const GetCursosToAlunos = (req, res) =>
  (async () => {
    let registro = await mdlAlunos.GetCursosToAlunos();
    res.json({ status: "ok", "registro": registro });
  })();

...
...

module.exports = {
  GetAllAlunos,
  GetAlunoByID,
  GetCursosToAlunos,
```

```
  InsertAluno,
  UpdateAluno,
  DeleteAluno
};
```

3. No arquivo de *rotas*

```
// Arquivo: backend/routes/router.js

...
...

// Rotas de Alunos

routerApp.get("/getAllAlunos", appAlunos.GetAllAlunos);
routerApp.get("/getAlunoByID/:alunoid", appLogin.AutenticaJWT, appAlunos.GetAlunoByID);
routerApp.get("/getCursosToAlunos", appLogin.AutenticaJWT, appAlunos.GetCursosToAlunos); // Linha
  adicionada.
routerApp.post("/insertAluno", appLogin.AutenticaJWT, appAlunos.InsertAluno);
routerApp.put("/updateAluno/:alunoid", appLogin.AutenticaJWT, appAlunos.UpdateAluno);
routerApp.delete("/deleteAluno/:alunoid", appLogin.AutenticaJWT, appAlunos.DeleteAluno);
```

```
// Rotas de Cursos
...
...
```

4. No arquivo de *testes*

```
// Arquivo: backend/apps/alunos/tests/testAlunos.rest

...
...

### teste de buscar cursos para alunos
GET http://localhost:40000/getCursosToAlunos HTTP/1.1
content-type: application/json
Authorization: Bearer {{accessToken}}

...
...
```

Pronto, agora que a API de cursos está pronta, vamos implementar o Cadastro para o módulo de alunos. Para

isso, cria-se o arquivo *vwFormAlunos.js*. Note a **linha *var oper = new URLSearchParams (window.location.search).get('ope*** Ela atribui a varíavel *oper* o valor do parâmetro *oper* passado na *url* quando da abertura do formulário de aluno.

O arquivo *vwFormAlunos.js* terá o seguinte conteúdo:

```
// Arquivo: apps/alunos/vwFormAlunos.js

document.addEventListener('DOMContentLoaded', function() {
  if (!dw3IsLogged()) {
    return;
  }

  var form = document.getElementById('frmAlunos');

  if (!form) {
    return;
  }

  var oper = parametros.get('oper') || parametros.get('Oper');
  var servidorDw3 = form.dataset.servidorDw3;
  var btnInserir = document.getElementById('btnInserirAluno');

  inicializarSelectCursos();

  if (oper === 'Cr') {
    btnInserir.classList.remove('d-none'); // Torna o Botão visível.
    carregarCursosToAlunos(servidorDw3);
```

```
  }

  if (btnInserir) {
    btnInserir.addEventListener('click', function() {
      vwInsertAluno();
    });
  }
});

// Aqui é inicializado o componente Select2.js em branco
function inicializarSelectCursos() {
  $('#cursoid').select2({
    theme: 'bootstrap-5',
    placeholder: 'Selecione um curso',
    width: '100%'
  });
}

// Aqui é preenchido o componente Select2 com os cursos disponíveis
async function carregarCursosToAlunos(servidorDw3) {
  try {
    if (!servidorDw3) {
      throw new Error('Endereço do servidor backend nao configurado.');
    }

    var response = await fetch(servidorDw3 + '/getCursosToAlunos', {
```

```
      headers: dw3MontarHeadersAutenticacao() // Função que está no arquivo public/js/dw3frontend.js
    });

    if (!response.ok) {
      throw new Error('Não foi possivel carregar os cursos.');
    }

    var data = await response.json();

    if (data.auth === false) {
      throw new Error(data.message || 'Sessão expirada. Faca login novamente.');
    }

    if (data.status !== 'ok' || !Array.isArray(data.registro)) {
      throw new Error('Resposta invalida do servidor backend.');
    }

    preencherSelectCursos(data.registro);
  } catch (error) {
    alert(error.message || 'Erro ao carregar cursos.');
  }
}

function preencherSelectCursos(cursos) {
  var cursoSelect = document.getElementById('cursoid');
```

```
  if (!cursoSelect) {
    return;
  }

  cursoSelect.innerHTML = '<option value="">Selecione um curso</option>';

  cursos.forEach(function(curso) {
    var option = document.createElement('option');
    option.value = curso.cursoid;
    option.textContent = curso.descricao;
    cursoSelect.appendChild(option);
  });

  $('#cursoid').trigger('change');
}

async function vwInsertAluno() {
  var form = document.getElementById('frmAlunos');
  var btnInserir = document.getElementById('btnInserirAluno');

  try {
    if (!form.reportValidity()) {
      return;
    }

    var servidorDw3 = form.dataset.servidorDw3;
```

```
if (!servidorDw3) {
  throw new Error('Endereco do servidor backend nao configurado.');
}

if (btnInserir) {
  btnInserir.disabled = true;
}

var response = await fetch(servidorDw3 + '/insertAluno', {
  method: 'POST',
  headers: dw3MontarHeadersAutenticacao({
    'content-type': 'application/json'
  }),
  body: JSON.stringify(montarAlunoDoFormulario())
});

if (!response.ok) {
  throw new Error('Nao foi possivel inserir o aluno.');
}

var data = await response.json();

if (data.auth === false) {
  throw new Error(data.message || 'Sessao expirada. Faca login novamente.');
}
```

```
    if (data.status !== 'ok') {
      throw new Error(data.status || 'Nao foi possivel inserir o aluno.');
    }

    window.location.href = '/alunos';
  } catch (error) {
    alert(error.message || 'Erro ao inserir aluno.');
  } finally {
    if (btnInserir) {
      btnInserir.disabled = false;
    }
  }
}

function montarAlunoDoFormulario() {
  return {
    prontuario: document.getElementById('prontuario').value,
    nome: document.getElementById('nome').value,
    endereco: document.getElementById('endereco').value,
    rendafamiliar: document.getElementById('rendafamiliar').value,
    datanascimento: document.getElementById('datanascimento').value,
    cursoid: document.getElementById('cursoid').value,
    deleted: document.getElementById('deleted').value === 'true'
  };
}
```

Note que temos duas funções, *dw3MontarHeadersAutenticacao()* e *dw3OcultarBotao()*, que devem ser implementadas no arquivo *public/js/dw3frontend.js*. Como são funções que podem ser usadas por diversas outras funções mesmo em arquivos diferentes, é mais produtivo implementadar tudo em um único arquivo do que ficar copiando seu código em todos os lugares que forem necessárias.

Assim, o arquivo *dw3frontend.js* vai ficar com a seguinte implementação:

```
// arquivo: public/js/dw3frontend.js

function dw3IsLogged() {
  const isLogged = document.cookie
    .split(';')
    .some((cookie) => cookie.trim().startsWith('IsLogged='));

  if (!isLogged) {
    window.location.href = '/login';
    return false;
  }

  return true;
}



function dw3ClearIsLoggedCookie() {
  // Limpa o Cookie Islogged
  var isLogged = document.cookie
    .split(';')
    .some((cookie) => cookie.trim().startsWith('IsLogged='));

  if (isLogged) {
    document.cookie = 'IsLogged=; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT';
  }
```

```
}

function dw3MontarHeadersAutenticacao(headers) { // Função nova
  var token = localStorage.getItem('token');
  var finalHeaders = headers || {};

  if (token) {
    finalHeaders.Authorization = 'Bearer ' + token;
  }

  return finalHeaders;
}

function dw3OcultarBotao(id) { // Função nova
  var botao = document.getElementById(id);

  if (botao) {
    botao.classList.add('d-none');
  }
}
```

O arquivo *apps/alunos/vwFormAlunos.njk* também sofreu modificação. Foram adicionadas a importação dos arquivo *vwFormAlunos.js* e *public/js/dw3frontend.js*. Também, foi removido a iniciação do *Select2.js*, pois, agora, ele está no arquivo *vwFormAlunos.js*.

Acrescente, no final do arquivo, antes de fechar o endblock, as seguinte modificações no arquivo *vwFormAlu-*

nos.njk:

```
// Arquivo: apps/alunos/vwFormAlunos.njk

{% extends "layout.njk" %}


{% block content %}

  ...
  ...
  ...

  <script src="/js/dw3frontend.js"></script> <!-- Linha adicionada -->
  <script src="https://code.jquery.com/jquery-4.0.0.slim.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/select2@4.1.0-rc.0/dist/js/select2.min.js"></script>
  <script src="/apps/alunos/vwFormAlunos.js"></script> <!-- Linha adicionada -->
{% endblock %}
```

3.8.10 Implementando a busca de Aluno para visualizar os dados

Agora, vamos alterar o arquivo *apps/alunos/vwFormAlunos.js* para acrescentar a função *vwGetAlunoByID()* que irá recuperar os dados de um aluno conforme seu *ID*.

Primeiro, temos de alterar no arquivo *vwLstAlunos.js* a função *configurarOperacoesAlunos ()* para implementar a chamada do formulário de alunos passando o parâmetro *oper=Re* e *alunoId=*.

```
// Arquivo: apps/alunos/vwLstAlunos.js

  ...

  function configurarOperacoesAlunos(alertBox) {
    var tabela = document.getElementById('tblAlunos');

    ...
    ...
    ...



    var action = botao.dataset.action;
    var alunoId = botao.dataset.alunoid;

    if (action === 'visualizar') {
      window.location.href = '/alunos/form?oper=Re&alunoId=' + encodeURIComponent(alunoId); // Linha
   alterada
      return;
    }

    ...
    ...
```

Feito isso, agora, vamos alterar o arquivo *vwFormAlunos.js* para analisar os parâmetros da URL e buscar as

informações do registro selecionado no servidor *backend*.

```
// Arquivo: apps/alunos/vwFormAlunos.js

document.addEventListener('DOMContentLoaded', function() {

  ...
  ...

  if (oper === 'Cr') {
    btnInserir.classList.remove('d-none'); // Torna o Botão visível.
    carregarCursosToAlunos(servidorDw3);
  }

  if (oper === 'Re') { // Código Adicionado.
    vwGetAlunoByID();
  }

  ...
  ...


  // Função adicionada.
  async function vwGetAlunoByID() {
  var form = document.getElementById('frmAlunos');
```

```
try {
  var parametros = new URLSearchParams(window.location.search);
  var alunoId = parametros.get('alunoId');
  var servidorDw3 = form.dataset.servidorDw3;

  if (!alunoId || !/^\d+$/.test(alunoId) || Number(alunoId) <= 0) {
    throw new Error('ID do aluno invalido.');
  }

  if (!servidorDw3) {
    throw new Error('Endereco do servidor backend nao configurado.');
  }

  var response = await fetch(servidorDw3 + '/getAlunoByID/' + encodeURIComponent(alunoId), {
    headers: dw3MontarHeadersAutenticacao({
      'content-type': 'application/json'
    })
  });

  if (!response.ok) {
    throw new Error('Nao foi possivel carregar o aluno.');
  }

  var data = await response.json();

  if (data.auth === false) {
```

```
      throw new Error(data.message || 'Sessao expirada. Faca login novamente.');
    }

    if (data.status !== 'ok' || !Array.isArray(data.registro)) {
      throw new Error('Resposta invalida do servidor backend.');
    }

    var aluno = data.registro[0];

    if (!aluno) {
      throw new Error('Aluno nao encontrado.');
    }

    document.getElementById('alunoid').value = aluno.alunoid;
    document.getElementById('prontuario').value = aluno.prontuario ?? '';
    document.getElementById('nome').value = aluno.nome ?? '';
    document.getElementById('endereco').value = aluno.endereco ?? '';
    document.getElementById('rendafamiliar').value = aluno.rendafamiliar ?? '';
    document.getElementById('datanascimento').value = aluno.datanascimento ?
   aluno.datanascimento.slice(0, 10) : '';
    document.getElementById('deleted').value = String(aluno.deleted === true);

    await carregarCursosToAlunos(servidorDw3); // Carrega o component Select2.
    $('#cursoid').val(aluno.cursoid).trigger('change');
  } catch (error) {
    alert(error.message || 'Erro ao carregar aluno.');
```

```
  }
}
```

3.8.11 Implementando a atualização de dados do Aluno

O processo de atualização é semelhante com o de busca. Vamos alterar o arquivo *apps/alunos/vwFormAlunos.js* para acrescentar a função *vwUpdateAluno()* que irá atualizar os dados de um aluno conforme seu *ID*.

Primeiro, temos de alterar no arquivo *vwLstAlunos.js* a função *configurarOperacoesAlunos ()* para implementar a chamada do formulário de alunos passando o parâmetro *oper=Up* e *alunoId=*.

```
// Arquivo: apps/alunos/vwLstAlunos.js

  ...

  function configurarOperacoesAlunos(alertBox) {
    var tabela = document.getElementById('tblAlunos');

    ...
    ...
    ...



    var action = botao.dataset.action;
    var alunoId = botao.dataset.alunoid;
```

```
    if (action === 'visualizar') {
      window.location.href = '/alunos/form?oper=Re&alunoId=' + encodeURIComponent(alunoId);
      return;
    }


    if (action === 'atualizar') {
      window.location.href = '/alunos/form?oper=Up&alunoId=' + encodeURIComponent(alunoId); // Linha
   alterada
      return;
    }

    ...
    ...
```

Feito isso, agora, vamos alterar o arquivo *vwFormAlunos.js* para analisar os parâmetros da URL, buscar as informações do registro selecionado no servidor *backend* e **enviar os dados atualizados**.

```
// Arquivo: apps/alunos/vwFormAlunos.js

document.addEventListener('DOMContentLoaded', function() {

  ...
  ...
```

```
if (oper === 'Cr') {
  btnInserir.classList.remove('d-none'); // Torna o Botão visível.
  carregarCursosToAlunos(servidorDw3);
}

if (oper === 'Re') {
  vwGetAlunoByID();
}



if (oper === 'Up') { // Código adicionado
  btnAtualizar.classList.remove('d-none');
  btnAtualizar.disabled = !(await vwGetAlunoByID());
  btnAtualizar.addEventListener('click', function() {
    vwUpdateAluno();
  });
}

...
...



// Função adicionada
async function vwUpdateAluno() {
var form = document.getElementById('frmAlunos');
var btnAtualizar = document.getElementById('btnAtualizarAluno');
```

```
if (btnAtualizar.disabled || !form.reportValidity()) {
  return;
}

try {
  var alunoId = new URLSearchParams(window.location.search).get('alunoId');
  var servidorDw3 = form.dataset.servidorDw3;

  if (!alunoId || !/^\d+$/.test(alunoId) || Number(alunoId) <= 0) {
    throw new Error('ID do aluno invalido.');
  }

  if (!servidorDw3) {
    throw new Error('Endereco do servidor backend nao configurado.');
  }

  btnAtualizar.disabled = true;

  var aluno = montarAlunoDoFormulario();
  aluno.rendafamiliar = Number(aluno.rendafamiliar);
  aluno.cursoid = Number(aluno.cursoid);

  var response = await fetch(servidorDw3 + '/updateAluno/' + encodeURIComponent(alunoId), {
    method: 'PUT',
    headers: dw3MontarHeadersAutenticacao({
```

```
        'content-type': 'application/json'
      }),
      body: JSON.stringify(aluno)
    });

    if (!response.ok) {
      throw new Error('Nao foi possivel atualizar o aluno.');
    }

    var data = await response.json();

    if (data.auth === false) {
      throw new Error(data.message || 'Sessao expirada. Faca login novamente.');
    }

    if (data.status !== 'ok') {
      throw new Error(data.status || 'Nao foi possivel atualizar o aluno.');
    }

    if (data.linhasAfetadas !== 1) {
      throw new Error('Nenhum aluno foi atualizado.');
    }

    alert('Aluno atualizado com sucesso.');
  } catch (error) {
    alert(error.message || 'Erro ao atualizar aluno.');
```

```
  } finally {
    btnAtualizar.disabled = false;
  }
}
```

3.8.12 Implementando a remoção do Aluno

O processo de remoção é semelhante aos de busca e atualização. Vamos alterar o arquivo *apps/alunos/vwFormAlunos.js* para acrescentar a função *vwDeleteAluno()* que irá remover o aluno conforme seu *ID*.

Primeiro, temos de alterar no arquivo *vwLstAlunos.js* a função *configurarOperacoesAlunos ()* para implementar a chamada do formulário de alunos passando o parâmetro *oper=Up* e *alunoId=*.

```
// Arquivo: apps/alunos/vwLstAlunos.js

  ...

  function configurarOperacoesAlunos(alertBox) {
    var tabela = document.getElementById('tblAlunos');

    ...
    ...
    ...
```

```
var action = botao.dataset.action;
var alunoId = botao.dataset.alunoid;

if (action === 'visualizar') {
  window.location.href = '/alunos/form?oper=Re&alunoId=' + encodeURIComponent(alunoId);
  return;
}


if (action === 'atualizar') {
  window.location.href = '/alunos/form?oper=Up&alunoId=' + encodeURIComponent(alunoId); // Linha
alterada
  return;
}


if (action === 'remover') { // Código adicionado
  window.location.href = '/alunos/form?oper=De&alunoId=' + encodeURIComponent(alunoId);
}


...
...
```

Feito isso, agora, vamos alterar o arquivo *vwFormAlunos.js* para analisar os parâmetros da URL, buscar as

informações do registro selecionado no servidor *backend* e **enviar os dados atualizados**.

```
// Arquivo: apps/alunos/vwFormAlunos.js

document.addEventListener('DOMContentLoaded', function() {

  ...
  ...

  if (oper === 'Cr') {
    btnInserir.classList.remove('d-none'); // Torna o Botão visível.
    carregarCursosToAlunos(servidorDw3);
  }

  ...
  ...

  if (oper === 'De') { //Rotina adicionada
    var btnRemover = document.getElementById('btnRemoverAluno');
    btnRemover.classList.remove('d-none');
    btnRemover.disabled = !(await vwGetAlunoByID());
    btnRemover.addEventListener('click', function() {
      vwDeleteAluno();
    });
  }
```

```
...
...


// Função adicionada
async function vwDeleteAluno() {

  var form = document.getElementById('frmAlunos');
  var btnRemover = document.getElementById('btnRemoverAluno');

  if (btnRemover.disabled) {
    return;
  }

  if (!window.confirm('Confirma a remoção deste aluno?')) {
    return;
  }

  try {
    var alunoId = document.getElementById('alunoid').value;
    var servidorDw3 = form.dataset.servidorDw3;

    if (!alunoId || !/^\d+$/.test(alunoId) || Number(alunoId) <= 0) {
      throw new Error('ID do aluno invalido.');
```

```
    }

    if (!servidorDw3) {
      throw new Error('Endereco do servidor backend nao configurado.');
    }

    btnRemover.disabled = true;

    var response = await fetch(servidorDw3 + '/deleteAluno/' + encodeURIComponent(alunoId), {
      method: 'DELETE',
      headers: dw3MontarHeadersAutenticacao()
    });

    if (!response.ok) {
      throw new Error('Nao foi possivel remover o aluno.');
    }

    var data = await response.json();

    if (data.auth === false) {
      throw new Error(data.message || 'Sessao expirada. Faca login novamente.');
    }

    if (data.status !== 'ok') {
      throw new Error(data.status || 'Nao foi possivel remover o aluno.');
    }
```

```
      if (data.linhasAfetadas !== 1) {
        throw new Error('Nenhum aluno foi removido.');
      }

      window.location.href = '/alunos';
    } catch (error) {
      alert(error.message || 'Erro ao remover aluno.');
    } finally {
      btnRemover.disabled = false;
    }

}
```

3.9 Implementando o módulo de Cursos

O desenvolvimento do módulo de *Cursos* possui a mesma sequência do de *Alunos*. Você deverá seguir a regra de:

item Crie o arquivo de lista em *./apps/módulo*.Crie uma **rota** em *./routes/módulo* que abra o formulário daquele módulo. Importe a rota no arquivo *./app.js*. Faça o teste para ver se o formulário é aberto. Crie o arquivo de formulário de campos em *./apps/módulo*.

Porém, aqui, não iremos colocar o código fonte, desta forma, você dverá usar o módulo de Alunos para dsenvolver as rotinas do módulo de Cursos.

3.10 Finalizando

Vimos como criar um servidor de *Front-End*. Este modelo pode ser utilizado para o desenvolvimento de aplicações reais e favorece a divisão de responsabilidades entre profissionais desenvolvedores de APIs e profissionais responsáveis por projetar a interface do sistema. Este modelo deixa o desenvolvimento produtivo, pois separa as competências dos profissionais permitindo que eles possam se especializar em suas áreas.

Capítulo 4

Deploy da Aplicação com Containers

4.1 Introdução

Durante o desenvolvimento, uma aplicação depende de uma versão específica do *Node.js*, de bibliotecas instaladas pelo NPM, de arquivos de configuração e de outros recursos do sistema operacional. Ao transferir a aplicação para outro computador, diferenças nesse ambiente podem provocar erros.

Um **container** empacota a aplicação e as dependências necessárias para sua execução em um ambiente isolado e padronizado. Desse modo, a mesma imagem pode ser utilizada no computador do desenvolvedor, no servidor de testes e no servidor de produção.

Neste capítulo, iremos criar imagens da aplicação *front-end*, cuja porta é 40100, e do *back-end*,alterando-se a porta para 40000 e adequando-se as variáveis de ambiente.

4.2 Conceitos fundamentais

4.2.1 Container

Um *container* é um processo isolado que compartilha o núcleo do sistema operacional do computador hospedeiro. Ele possui seu próprio sistema de arquivos, suas bibliotecas e sua configuração de rede. Em comparação com uma máquina virtual completa, normalmente utiliza menos recursos e inicia mais rapidamente, pois não precisa executar outro sistema operacional completo.

O isolamento não significa que os dados sejam permanentes. Quando um *container* é removido, os arquivos gravados somente em sua camada interna também são removidos. Dados que precisam ser preservados devem ser armazenados em volumes ou em serviços externos, como um SGBD.

4.2.2 Imagem

Uma **imagem** é um modelo imutável que contém os arquivos e as instruções necessárias para executar a aplicação. A partir de uma mesma imagem, podemos criar vários *containers*. Portanto, a imagem é o modelo e o *container* é uma instância em execução desse modelo.

4.2.3 Docker

O *Docker* é uma plataforma para construir, distribuir e executar *containers*. Neste capítulo, serão utilizados principalmente os comandos:

4. *docker build*: constrói uma imagem a partir de um *Dockerfile*;
   - *docker run*: cria e inicia um *container*;
   - *docker ps*: lista os *containers*;
   - *docker logs*: exibe as mensagens produzidas pela aplicação;
   - *docker stop* e *docker start*: interrompem e reiniciam um *container*.

Para verificar se o Docker está instalado e se o serviço está disponível, execute:

```
docker --version
docker info
```

4.3 Criando a imagem da aplicação

4.3.1 Arquivo dockerignore

O arquivo *.dockerignore* impede que arquivos desnecessários ou sensíveis sejam enviados para o contexto de construção. Crie-o no mesmo diretório do *Dockerfile*:

```
## Criar o arquivo
touch .dockerignore
```

```
## Conteúdo

node_modules
.git
.env
.env.*
*.env
npm-debug.log*
coverage
Dockerfile
.dockerignore
```

O arquivo de variáveis de ambiente não deve fazer parte da imagem. Seus valores serão fornecidos somente na criação do *container*.

4.3.2 Arquivo Dockerfile do servidor *BackEnd*

O arquivo *Dockerfile* descreve as etapas de construção da imagem. Na raiz do diretório *dw3backend*, crie um arquivo chamado *Dockerfile*, sem extensão, com o seguinte conteúdo:

```
## Diretório: dw3backend/

FROM node:24-bookworm-slim
```

```
ENV NODE_ENV=production

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY --chown=node:node . .

USER node

EXPOSE 40000

CMD ["node", "app.js"]
```

Listing 4.1: Dockerfile do servidor back-end.

As instruções possuem as seguintes finalidades:

- comentário;
- *FROM* define a imagem base com o Node.js;
- *WORKDIR* define o diretório de trabalho dentro da imagem;
- *COPY* copia os arquivos necessários para a imagem;

- *RUN* executa um comando durante a construção;
- *ENV* define uma variável de ambiente padrão;
- *EXPOSE* documenta a porta utilizada pela aplicação;
- *CMD* define o comando executado ao iniciar o *container*.

O comando *npm ci* utiliza o arquivo *package-lock.json* para instalar as versões registradas das dependências. Portanto, esse arquivo deve estar no repositório.

4.3.3 Criando a imagem do servidor *BackEnd*

Para criarmos a imagem do servidor *backend*, execute o comando a seguir no mesmo diretório em que se localiza o arquivo *Dockerfile*:

```
docker build -t dw3backend_img:1.0 .
```

Após criado a imagem, verifique com

```
docker images
```

4.3.4 Executando a imagem do *BackEnd*

No diretório em que se encontra o arquivo *.env*, execute **observando com atenção a situação do Postgres**:

```
## Postgres rodando em docker
docker run -d \
  --name dw3backend_ctner \
  --env-file .env \
  -p 40000:40000 \
  --restart unless-stopped \
  dw3backend_img:1.0


## Postgres rodando local
docker run -d \
  --name dw3backend_ctner \
  --add-host host.docker.internal:host-gateway \
  --env-file .env \
  -p 40000:40000 \
  --restart unless-stopped \
  dw3backend_img:1.0

Depois, altere o DB_HOST no arquivo .env para:
DB_HOST=host.docker.internal
```

Nesse comando:

- *-d* executa o *container* em segundo plano;

- *–name* atribui um nome ao *container*;
- add-host host.docker.internal:host-gateway : Cria uma regra de mapeamento de rede interna. Ela permite que o container acesse serviços que estão rodando diretamente na sua máquina hospedeira (host) através do endereço host.docker.internal. O host-gateway resolve automaticamente para o IP correto do gateway do Docker no seu sistema operacional.
- *–env-file .env*: carrega as configurações do backend.
- *-p 40000:40000*: mapeia a porta do contêiner para sua máquina.
- *–restart unless-stopped*: reinicia o contêiner automaticamente, exceto quando você o para manualmente.

```
docker ps
docker logs dw3backend_ctner
```

Para interromper e iniciar novamente o *container*, execute:

```
docker stop dw3backend_ctner
docker start dw3backend_ctner
```

Quando o *container* não for mais necessário, primeiro interrompa sua execução e depois o **remova**:

```
docker stop dw3backend_ctner
docker rm dw3backend_ctner
```

Esses comandos removem o *container*, mas mantêm a imagem *dw3backend_img*, que poderá ser utilizada novamente.

4.3.5 Arquivo Dockerfile do servidor *FrontEnd*

Na raiz do diretório *dw3frontend*, crie um arquivo chamado *Dockerfile*, sem extensão, com o seguinte conteúdo:

```
## Diretório: dw3frontend/

FROM node:24-bookworm-slim

ENV NODE_ENV=production

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY --chown=node:node . .

USER node

EXPOSE 40100

CMD ["node", "app.js"]
```

4.3.6 Criando a imagem do servidor *FrontEnd*

Para criarmos a imagem do servidor *frontend*, execute o comando a seguir no mesmo diretório em que se localiza o arquivo *Dockerfile*:

```
docker build -t dw3frontend_img:1.0 .
```

Após criado a imagem, verifique com

```
docker images
```

4.3.7 Executando a imagem do servidor *FrontEnd*

No diretório em que se encontra o arquivo *dw3frontend.env*, execute:

```
docker run -d \
  --name dw3frontend_ctner \
  --env-file dw3frontend.env \
  -p 40100:40100 \
  --restart unless-stopped \
  dw3frontend_img:1.0
```

Abra o endereço *http://localhost:40100* no navegador. Para conferir a execução e visualizar as mensagens da aplicação, utilize:

```
docker ps
docker logs dw3frontend_ctner
```

Para interromper e iniciar novamente o *container*, execute:

```
docker stop dw3frontend_ctner
docker start dw3frontend_ctner
```

Quando o *container* não for mais necessário, primeiro interrompa sua execução e depois o remova:

```
docker stop dw3frontend_ctner
docker rm dw3frontend_ctner
```

Esses comandos removem o *container*, mas mantêm a imagem $dw3frontend_img$, $que pode ser utilizada novamente$.

4.4 Atualizando a aplicação

Após uma alteração no repositório, a imagem existente não é modificada automaticamente. É necessário obter o código atualizado, reconstruir a imagem e substituir o container*:*

```
docker build -t dw3frontend:1.1 .
docker stop dw3frontend_ctner
docker rm dw3frontend_ctner
docker run -d --name dw3frontend_ctner -p 40100:40100 \
```

```
--env-file dw3frontend.env dw3frontend_img:1.1
```

O uso de uma nova etiqueta, como 1.1*, permite identificar exatamente a versão implantada e facilita o retorno a uma versão anterior.*

4.5 Removendo uma imagem

Quando uma imagem não for mais necessária, é possível ***removê-la*** *com o comando abaixo, porém, não pode haver* containers *da imagem em execução. Você deve primeiro parar o* container *e depois remover a imagem.*

```
docker rmi dw3frontend_img
```

4.6 Finalizando

Neste capítulo, vimos que uma imagem reúne a aplicação e suas dependências, enquanto um container *é uma instância isolada dessa imagem em execução. Também criamos um* Dockerfile*, construímos uma imagem com o código e executamos a aplicação com suas portas e variáveis de ambiente configuradas. Esse processo torna o ambiente de implantação mais padronizado e reproduzível.*